

Interview Question Bank

500 Questions with Answers, Based on the Resume of Yugdeep Parihar
Software Engineer | .NET Full-Stack | GenAI & Process Automation

Part A: HR & Behavioral Round (77 questions)

Part B: Technical Round (423 questions)

How to Use This Guide

Every question in this guide is linked to something on your resume: your Capgemini experience, the three automation tools (PTA Automation Tool, Audit Log Validator, Outlook Email Automation), your GenAI and prompt engineering work, the Online Banking Application, the Sign Language Detection project, and every skill in your Technical Skills section.

Personalize before your interview. Answers marked *(Customize)* or *(Answer honestly)* are sample answers. Replace them with your real details, such as notice period, CTC, the exact tech used in your tools and specific challenges you faced. Interviewers dig into project questions, so your answers must match what you actually did.

Don't memorize word for word. Understand each answer and say it in your own words. For behavioral questions, use the STAR method: Situation, Task, Action, Result, and end with a measurable result wherever possible (for example, the 98% effort reduction or the 18% accuracy improvement).

Prioritize. Focus first on questions about your own projects and the skills most relevant to the job description: .NET/C#, ASP.NET Core, SQL, Azure OpenAI and Power Automate. Then cover the rest.

Contents

Part A: HR & Behavioral Round

- Introduction & Background (Q1-Q10)
- Strengths, Weaknesses & Self-Awareness (Q11-Q20)
- Achievements & Experience (Q21-Q30)
- Behavioral (STAR) Questions (Q31-Q45)
- Work Style & Culture (Q46-Q55)
- Career Goals (Q56-Q61)
- Salary, Logistics & Closing (Q62-Q77)

Part B: Technical Round

- C# Fundamentals & OOP (Q78-Q127)
- SOLID & Design Patterns (Q128-Q137)
- .NET Framework & Runtime (Q138-Q146)
- ASP.NET Core & Web API (Q147-Q186)
- Entity Framework & Data Access (Q187-Q197)
- Angular & Front End (Q198-Q232)
- SQL & Databases (Q233-Q277)
- Generative AI, Azure OpenAI & Prompt Engineering (Q278-Q328)
- Automation, Power Automate & My Automation Tools (Q329-Q360)
- Unit Testing, NUnit & Code Quality (Q361-Q380)
- DevOps, Git, CI/CD & Cloud (Q381-Q424)
- Python, Machine Learning & Deep Learning (Q425-Q465)
- Project Deep-Dive, SDLC, Agile & Problem Solving (Q466-Q500)

Part A: HR & Behavioral Round

Introduction & Background

Q1. Tell me about yourself.

Answer: I'm Yugdeep Parihar, a Software Developer at Capgemini with about 1.5 years of experience, including my internship. I completed my Bachelor's in Computer Science (Hons.) from GLA University with a CPI of 8.29. At Capgemini I work on .NET full-stack development and Generative AI solutions using Azure OpenAI. I also act as my team's Automation Lead: I built a PTA automation tool that cut a 1.25-hour manual process to about 1 minute, an Audit Log Validator, and an Outlook email automation using Power Automate. I enjoy solving real problems that save my team time, and I'm now looking for a role where I can grow in full-stack development, GenAI and automation.

Q2. Walk me through your resume.

Answer: I graduated in Computer Science from GLA University in 2025. I joined Capgemini as an intern in April 2025, where I trained in C#, .NET and Angular and built an E-Banking application. In June 2025 I became a Software Developer. Since then I've developed GenAI solutions on Azure OpenAI, improved model accuracy through prompt engineering, and led automation initiatives: the PTA tool, the Audit Log Validator and an Outlook automation. Outside work I built an Online Banking application and a Sign Language Detection system using CNNs.

Q3. Why did you choose Computer Science?

Answer: I've always enjoyed logical problem solving and building things. Computer Science gave me a way to turn ideas into working tools that people actually use. Seeing my automation tools save my team hours of work confirms I made the right choice.

Q4. How would you describe your career so far in one line?

Answer: A developer who started with .NET full-stack work and quickly grew into leading automation and GenAI initiatives that deliver measurable time savings.

Q5. What do you know about our company?

Answer: *(Customize before each interview.)* Research the company's products, clients, tech stack and recent news. Then connect it to your skills, for example: "I noticed you use .NET and Azure and are investing in AI, which matches my experience with ASP.NET Core and Azure OpenAI."

Q6. Why do you want to join our company?

Answer: I'm looking for a place where I can work on challenging products and grow my skills in full-stack .NET, GenAI and automation. Your work in [area] and your use of [technology] align closely with what I've been doing, so I believe I can contribute from day one while continuing to learn.

Q7. Why are you looking for a change after 1.5 years?

Answer: Capgemini gave me a strong foundation and I'm grateful for it. I've grown quickly, especially by leading automation work, and I now want a role with more product ownership, larger technical challenges and faster growth in GenAI and full-stack development. It's a move toward growth, not away from anything negative.

Q8. What are you currently working on at Capgemini?

Answer: I work on .NET-based development and GenAI solutions using Azure OpenAI APIs. I also lead automation within my project team, identifying manual processes and building tools such as the PTA automation tool, the Audit Log Validator and an Outlook email automation with Power Automate.

Q9. What is your notice period?

Answer: (*Answer honestly.*) My notice period is [X] days. I'm open to discussing an early release if the joining timeline requires it.

Q10. Are you currently holding any offers?

Answer: (*Answer honestly.*) Be truthful. If you have an offer, mention it politely without using it aggressively. If not, say you're actively interviewing and this role is a strong fit.

Strengths, Weaknesses & Self-Awareness

Q11. What are your strengths?

Answer: First, problem solving with an automation mindset: I notice repetitive work and build tools to remove it, like the PTA tool that cut 98% of manual effort. Second, quick learning: I went from an intern to leading automation initiatives within about a year. Third, ownership: I take work from idea to delivery, including testing and rollout.

Q12. What is your biggest weakness?

Answer: Earlier, I tended to take on too much myself instead of delegating, because I wanted things done perfectly. As I started leading automation initiatives, I learned to break work down, share tasks and trust teammates. I'm still improving, but it's made me more effective.

Q13. What is one skill you are currently improving?

Answer: I'm deepening my knowledge of system design and cloud-native architecture, especially building scalable services on Azure, because I want to design larger systems, not only individual features.

Q14. How would your manager describe you?

Answer: As reliable, proactive and solution-oriented. My manager has seen me identify inefficiencies and deliver tools that save the team significant time without being asked.

Q15. How would your teammates describe you?

Answer: As approachable and helpful. I share what I build, explain how the tools work, and help teammates when they're stuck.

Q16. What motivates you?

Answer: Seeing my work make a real difference. When a tool I built turns a 75-minute task into a 1-minute task, and teammates spend that time on better work, that's very motivating.

Q17. What demotivates you?

Answer: Repetitive work that could be automated but isn't. The good thing is that this frustration usually turns into an idea for a tool.

Q18. Are you a team player or do you prefer working alone?

Answer: I'm comfortable with both, but I get the best results in a team. My automation tools came from understanding teammates' pain points, and their feedback made the tools better.

Q19. How do you handle criticism?

Answer: I treat it as useful information. In code reviews, I listen, ask questions to understand the concern, and fix the issue. Feedback on my tools has directly improved their accuracy and usability.

Q20. What makes you unique compared to other candidates?

Answer: Beyond regular development, I've proven I can spot inefficiencies and fix them with measurable results, such as a 98% effort reduction. I combine .NET full-stack skills, GenAI experience and automation, which is a useful mix.

Achievements & Experience

Q21. What is your biggest achievement so far?

Answer: Building the PTA automation tool. A process that took about 1.25 hours of manual work now takes about 1 minute, a 98% reduction, and built-in validation makes the output accurate. It saves the team time every time the task runs.

Q22. Tell me about a time you went beyond your assigned work.

Answer: I noticed my team spending a lot of time on a manual PTA process that wasn't in my assigned scope. I proposed an automation idea, built the tool, added validation, and rolled it out. It cut the effort from 1.25 hours to about 1 minute.

Q23. Describe a project you are most proud of.

Answer: The PTA automation tool, for its direct impact. The Audit Log Validator is a close second, because it lets us verify the whole team's work in minutes instead of a long manual review.

Q24. What does being an "Automation Lead" mean in your team?

Answer: I identify manual, repetitive processes, propose automation ideas, and lead the initiatives from idea to delivery: requirement analysis, development, testing and rollout. I'm not the overall project lead, but I own the automation work within the project.

Q25. How did you come up with the idea for the PTA tool?

Answer: I observed how much time the team spent on the task and how error-prone it was. I mapped out the steps, found that most were rule-based, and realized they could be automated with validation checks.

Q26. How did you convince your team or manager to adopt your automation idea?

Answer: I showed the time being spent, built a small working prototype, and demonstrated the time saved. Seeing the before-and-after numbers made the value clear, and adoption was easy.

Q27. How do you measure the success of your work?

Answer: Through measurable outcomes: time saved, reduced errors and adoption by the team. For example, the PTA tool reduced effort by 98%, and prompt engineering improved GenAI accuracy by 18%.

Q28. Tell me about a time you improved a process.

Answer: With the Outlook email automation. The team manually read emails and copied details into trackers. I built a Power Automate flow that captures the details automatically into structured data, which removed manual data entry and made the data available for analysis.

Q29. Have you ever mentored anyone?

Answer: Yes. I helped newer teammates understand our automation tools and supported them with code and debugging questions. I also participate in code reviews, which is a form of mentoring too.

Q30. What did you learn from your internship?

Answer: I learned C#, .NET, ASP.NET and Angular hands-on, built an E-Banking application, and understood how real software is developed following the SDLC. Most importantly, I learned how to work in a professional team.

Behavioral (STAR) Questions

Q31. Tell me about a time you faced a tight deadline.

Answer: (Use STAR.) Situation: a deliverable was due sooner than planned. Task: finish without reducing quality. Action: I broke the work into small tasks, prioritized the critical ones, communicated progress daily and used automation to speed up repetitive steps. Result: we delivered on time with no quality issues.

Q32. Tell me about a time you made a mistake.

Answer: Early on, I pushed a change without testing an edge case, and it caused an issue in testing. I owned it immediately, fixed it, and added unit tests for that scenario. Since then I always write tests for edge cases before submitting code.

Q33. Tell me about a conflict with a teammate and how you resolved it.

Answer: A teammate and I disagreed on how to structure a feature. Instead of arguing, we listed the pros and cons of both approaches and discussed them with data. We chose a combined approach, and the discussion improved the final design.

Q34. Describe a situation where you had to learn something quickly.

Answer: When I started on GenAI work, I had to learn Azure OpenAI APIs and prompt engineering quickly. I studied the documentation, built small experiments, and iterated on prompts. Within a short time I improved model accuracy by 18%.

Q35. Tell me about a time you handled a difficult stakeholder.

Answer: A stakeholder kept changing requirements. I set up a quick call, clarified priorities, documented agreed requirements in Jira, and shared progress regularly. That reduced surprises and built trust.

Q36. Tell me about a time you failed.

Answer: One early version of an automation tool didn't handle some unusual input formats and produced wrong output. I treated it as a lesson: I collected those cases, added validation logic, and tested more thoroughly. The next version was far more reliable.

Q37. Describe a time you took initiative.

Answer: Proposing and building the automation tools. Nobody assigned them to me. I saw the problem, built a solution, and it became part of the team's workflow.

Q38. Tell me about a time you worked under pressure.

Answer: During a release, several issues came up at once. I stayed calm, prioritized by impact, fixed the critical ones first and kept the team informed. We met the release date.

Q39. Tell me about a time you disagreed with your manager.

Answer: I suggested automating a task my manager felt wasn't worth the effort. I respected their view but showed the time spent per week and a quick prototype. With the data, my manager agreed, and the tool proved valuable.

Q40. Tell me about a time you helped a teammate.

Answer: A teammate was stuck on a bug before a deadline. I paired with them, debugged step by step, and we fixed it together. I also shared a tip to avoid the same issue in future.

Q41. Give an example of when you handled multiple tasks at once.

Answer: I was working on GenAI tasks while building an automation tool. I used Jira to track tasks, prioritized by deadlines and impact, and set focused time blocks for each. Both were delivered on time.

Q42. Tell me about a time you had to explain a technical concept to a non-technical person.

Answer: I explained the PTA tool to a non-technical stakeholder by comparing it to a machine that does the same steps a person does, only much faster and with built-in checking. I focused on time saved rather than code.

Q43. Describe a time you received negative feedback.

Answer: In a code review, I was told my code was hard to read. I accepted it, refactored with clearer names and smaller methods, and now I focus on readability from the start.

Q44. Tell me about a decision you made without complete information.

Answer: While designing validation for a tool, I didn't have every possible input format. I built validation for known cases, logged anything unexpected, and updated rules as new cases appeared. That let us launch quickly and improve safely.

Q45. Tell me about a time you had to adapt to change.

Answer: Moving from Pune to Noida and shifting focus to GenAI work required adapting quickly. I focused on learning the new domain and building relationships with the team, and I became productive quickly.

Work Style & Culture

Q46. How do you prioritize your work?

Answer: By impact and urgency. I list tasks in Jira, tackle blockers and deadline-critical items first, and schedule improvement work, like automation, in dedicated time.

Q47. How do you stay updated with technology?

Answer: I follow official documentation and blogs from Microsoft and Azure, experiment with new tools like GenAI APIs, and try applying what I learn to real problems at work.

Q48. How do you handle stress?

Answer: I break big problems into small steps, focus on what I can control, and communicate early if I see risks. Solving the first piece usually reduces the stress.

Q49. What kind of work environment do you prefer?

Answer: A collaborative environment where people share knowledge, ideas are welcome regardless of seniority, and there's room to learn and take ownership.

Q50. How do you handle repetitive tasks?

Answer: I do them properly, but I also look at whether they can be automated. Several of my tools started exactly this way.

Q51. What do you do when you don't know the answer to a problem?

Answer: I research documentation, break the problem down, try small experiments, and if I'm still stuck after a reasonable time, I ask a colleague with a clear summary of what I've already tried.

Q52. How do you ensure quality in your work?

Answer: I write unit tests, validate edge cases, participate in code reviews, and build validation into my tools so errors are caught automatically.

Q53. Are you comfortable working in Agile teams?

Answer: Yes. I work with Agile practices: sprint tasks, stand-ups and Jira tracking. I like the short feedback cycles.

Q54. How do you handle working with people from different backgrounds?

Answer: I stay respectful, listen actively and focus on shared goals. Different perspectives usually lead to better solutions.

Q55. What would you do if you were assigned a task outside your skill set?

Answer: I'd accept it as a learning opportunity, learn the basics quickly, ask for guidance where needed, and communicate realistic timelines.

Career Goals

Q56. Where do you see yourself in 5 years?

Answer: As a senior engineer or technical lead who designs scalable full-stack and AI-powered systems, leads a team, and drives automation and innovation across projects.

Q57. What are your short-term goals?

Answer: To join a team where I can work on impactful products, strengthen my full-stack and cloud skills, and contribute to GenAI and automation work from early on.

Q58. What are your long-term goals?

Answer: To become a technical leader who designs systems that solve real business problems and mentors other engineers.

Q59. Do you plan to pursue higher studies?

Answer: Right now I want to focus on industry experience. I prefer learning through certifications and real projects, such as Azure and AI certifications.

Q60. Would you be interested in a management role in the future?

Answer: Yes, eventually. I've enjoyed leading automation initiatives, and I'd like to grow into technical leadership, but I want to build strong technical depth first.

Q61. What kind of projects do you want to work on?

Answer: Full-stack .NET applications, GenAI solutions, and automation that creates measurable impact for users and teams.

Salary, Logistics & Closing

Q62. What are your salary expectations?

Answer: Based on my 1.5 years of experience, my skills in .NET, GenAI and automation, and market standards for this role, I'm expecting [range]. I'm open to discussion based on the overall package and growth opportunities.

Q63. Why should we hire you?

Answer: I bring hands-on .NET full-stack and GenAI experience and a proven track record of measurable impact, like a 98% reduction in manual effort through automation. I learn quickly, take ownership, and consistently look for ways to make the team more effective.

Q64. Are you willing to relocate?

Answer: *(Answer honestly.)* Yes, I'm open to relocating for the right opportunity. I've already worked in Navi Mumbai, Pune and Noida.

Q65. Are you comfortable with shift timings or working from the office?

Answer: *(Answer honestly.)* State your preference clearly, and show flexibility where possible.

Q66. Can you work under minimal supervision?

Answer: Yes. My automation tools were self-driven: I identified problems, designed solutions and delivered them independently, while keeping my manager informed.

Q67. What will you bring to our team in the first 90 days?

Answer: In the first month, I'll learn the codebase, processes and team. In the second, I'll deliver assigned features independently. By the third, I aim to contribute improvements such as automation or better testing.

Q68. What is your current CTC?

Answer: *(Answer honestly.)* State your current CTC clearly. Recruiters often verify it through documents.

Q69. Why is there a gap or short duration in your experience?

Answer: There is no gap: I joined Capgemini as an intern in April 2025 and moved into a full-time role in June 2025, and I've been there since.

Q70. Why did you move from Pune to Noida?

Answer: *(Answer honestly.)* For example: "It was a project or location change within Capgemini, and I adapted quickly to the new team."

Q71. What do you like most about your current job?

Answer: The freedom to identify problems and solve them. Leading automation work has shown me how much impact a single tool can have.

Q72. What do you dislike about your current job?

Answer: I don't dislike anything major. I'd simply like more exposure to larger-scale product development and advanced GenAI work, which is part of why I'm exploring this role.

Q73. How do you handle rejection?

Answer: I ask for feedback where possible, identify what I can improve, and treat it as preparation for the next opportunity.

Q74. What would you do if you disagreed with a company policy?

Answer: I'd follow it while raising my concern respectfully through the right channel, backed by reasons or data.

Q75. How do you define success?

Answer: Delivering work that creates real, measurable value and continuously becoming better than I was before.

Q76. If you get this job, how will you keep improving?

Answer: Through regular feedback, learning from senior engineers, earning relevant certifications, and taking ownership of challenging tasks.

Q77. Do you have any questions for us?

Answer: Always ask two or three. For example: "What does success look like in this role in the first six months?", "What technologies and projects would I work on?" and "How does the team support learning and growth?"

Part B: Technical Round

C# Fundamentals & OOP

Q78. What is C#?

Answer: C# is a modern, object-oriented, type-safe programming language developed by Microsoft that runs on .NET. It's used to build web apps, APIs, desktop apps, cloud services and more.

Q79. What are the four pillars of OOP?

Answer: Encapsulation (hiding data behind methods), Abstraction (exposing only essential features), Inheritance (reusing code from a base class) and Polymorphism (one interface with many implementations).

Q80. What is encapsulation? Give an example.

Answer: Encapsulation bundles data and the methods that use it, and restricts direct access to the data. For example, a BankAccount class keeps balance private and exposes Deposit() and Withdraw() methods that validate the amount before changing it.

Q81. What is abstraction?

Answer: Abstraction hides implementation details and shows only what's necessary. In C#, it's achieved with abstract classes and interfaces. For example, an IPaymentService interface defines Pay() without saying how payment is processed.

Q82. What is inheritance? Does C# support multiple inheritance?

Answer: Inheritance lets a class reuse and extend another class's members. C# does not support multiple inheritance of classes, to avoid the diamond problem, but a class can implement multiple interfaces.

Q83. What is polymorphism? What are its types?

Answer: Polymorphism lets one method behave differently based on context. Compile-time (static) polymorphism is method overloading. Run-time (dynamic) polymorphism is method overriding using virtual and override.

Q84. What is the difference between method overloading and overriding?

Answer: Overloading: the same method name with different parameters in the same class, resolved at compile time. Overriding: a derived class provides its own implementation of a virtual or abstract base method, resolved at run time.

Q85. What is the difference between an abstract class and an interface?

Answer: An abstract class can have fields, constructors, and both implemented and abstract methods, and a class can inherit only one. An interface defines a contract (since C# 8 it can have default implementations), has no instance fields, and a class can implement many. Use an abstract class for a shared base, and an interface for capabilities.

Q86. What is a sealed class?

Answer: A sealed class cannot be inherited. It's used to prevent further derivation for security or design reasons. For example, the string class is sealed.

Q87. What is a static class?

Answer: A static class cannot be instantiated and contains only static members. It's used for utility or helper methods, like Math.

Q88. What is the difference between a class and a struct?

Answer: A class is a reference type stored on the heap and supports inheritance. A struct is a value type, typically stored on the stack, and cannot inherit from another struct or class. Structs are good for small, immutable data like points or coordinates.

Q89. What are value types and reference types?

Answer: Value types (int, bool, struct) hold data directly, and a copy is made on assignment. Reference types (class, string, arrays) hold a reference to data on the heap, so two variables can point to the same object.

Q90. What is boxing and unboxing?

Answer: Boxing converts a value type into an object (reference type) on the heap. Unboxing converts it back. Both have performance costs, which is why generics are preferred.

Q91. What are access modifiers in C#?

Answer: public (accessible everywhere), private (within the class), protected (class and derived classes), internal (within the assembly), protected internal (assembly or derived classes), and private protected (derived classes within the same assembly).

Q92. What is the difference between const and readonly?

Answer: const is a compile-time constant and must be assigned at declaration. readonly can be assigned at declaration or in the constructor, so its value can be set at run time.

Q93. What is the difference between ref and out?

Answer: Both pass arguments by reference. A ref parameter must be initialized before passing. An out parameter doesn't need initialization but must be assigned inside the method.

Q94. What is the difference between == and Equals()?

Answer: For reference types, == checks reference equality by default, while Equals() can be overridden to check value equality. For strings, both compare values because string overloads ==.

Q95. Why are strings immutable in C#? What is StringBuilder?

Answer: Strings are immutable so they're thread-safe and can be safely shared and interned. Every modification creates a new string. StringBuilder is a mutable buffer that's far more efficient for many concatenations, for example in loops.

Q96. What is a constructor? What types exist?

Answer: A constructor initializes an object when it's created. Types include default, parameterized, copy (user-defined), static (runs once per type) and private (prevents external instantiation, used in the Singleton pattern).

Q97. What is a destructor or finalizer?

Answer: A finalizer (~ClassName) is called by the garbage collector before an object is reclaimed. It's rarely needed. The IDisposable pattern is preferred for releasing resources.

Q98. What is garbage collection in .NET?

Answer: The GC automatically frees memory of objects that are no longer reachable. It uses generations (0, 1 and 2): short-lived objects are collected frequently, and long-lived ones less often, which improves performance.

Q99. What is IDisposable and the using statement?

Answer: IDisposable defines Dispose() to release unmanaged resources like file handles or database connections. The using statement calls Dispose() automatically at the end of the scope, even if an exception occurs.

Q100. What is exception handling in C#?

Answer: It uses try (code that may fail), catch (handle specific exceptions), finally (always runs, for cleanup) and throw (raise an exception). Catch specific exceptions first and avoid swallowing errors silently.

Q101. What is the difference between throw and throw ex?

Answer: throw rethrows the exception and preserves the original stack trace. throw ex resets the stack trace, which makes debugging harder, so use throw.

Q102. How do you create a custom exception?

Answer: Create a class that inherits from Exception, add constructors that pass a message (and optionally an inner exception) to the base class, for example InsufficientBalanceException in a banking app.

Q103. What are generics?

Answer: Generics let you write type-safe, reusable code with type parameters, like List<T> or Repository<T>. They avoid boxing and casting and catch type errors at compile time.

Q104. What are collections in C#? Name a few.

Answer: Collections store groups of objects: List<T>, Dictionary<TKey,TValue>, HashSet<T>, Queue<T>, Stack<T> and LinkedList<T>. Concurrent versions like ConcurrentDictionary exist for multithreading.

Q105. What is the difference between an array and a List?

Answer: An array has a fixed size. A List<T> resizes dynamically and provides methods like Add, Remove and Find.

Q106. What is the difference between IEnumerable and IQueryable?

Answer: IEnumerable executes queries in memory (LINQ to Objects). IQueryable builds an expression tree that's translated and executed at the data source, like SQL via Entity Framework, so filtering happens in the database. That's more efficient for large data.

Q107. What is LINQ?

Answer: Language Integrated Query lets you query collections, databases and XML using C# syntax, with methods like Where, Select, OrderBy, GroupBy and Join.

Q108. What is deferred execution in LINQ?

Answer: A LINQ query isn't executed when defined but when it's enumerated (for example with foreach, ToList() or Count()). This allows query composition, but results reflect the data at enumeration time.

Q109. What is the difference between First() and FirstOrDefault()?

Answer: First() throws an exception if no element matches. FirstOrDefault() returns the default value (null for reference types) instead.

Q110. What are delegates?

Answer: A delegate is a type-safe reference to a method. Delegates enable callbacks and events. Built-in delegates include Action (no return value), Func (returns a value) and Predicate (returns bool).

Q111. What are events in C#?

Answer: Events are built on delegates and let a class notify subscribers when something happens, following the publisher-subscriber pattern. An example is a button click event.

Q112. What are lambda expressions?

Answer: Lambdas are anonymous functions written with =>, such as x => x * 2. They're used heavily in LINQ and as delegate arguments.

Q113. What are extension methods?

Answer: Static methods in a static class that add functionality to existing types without modifying them. The first parameter uses the this keyword. LINQ methods are extension methods.

Q114. What is async and await?

Answer: They enable asynchronous programming. An async method returns Task or Task<T>, and await pauses execution until the task completes without blocking the thread. This is essential for I/O-bound work like API calls and database queries, and it improves scalability.

Q115. What is the difference between Task and Thread?

Answer: A Thread is a low-level OS thread. A Task is a higher-level abstraction that represents an asynchronous operation and usually runs on the thread pool. It supports continuations, cancellation and async/await.

Q116. What is a deadlock and how do you avoid it in async code?

Answer: A deadlock happens when two operations wait on each other forever. In async code, blocking with `.Result` or `.Wait()` on async methods can deadlock. Avoid it by using `async` all the way and using `ConfigureAwait(false)` in library code.

Q117. What is the lock keyword?

Answer: `lock` ensures only one thread executes a block of code at a time, preventing race conditions on shared data.

Q118. What are nullable types?

Answer: Nullable value types (`int?` or `Nullable<int>`) can hold null. Nullable reference types (`C# 8+`) help catch possible null reference errors at compile time.

Q119. What are the ?? and ?. operators?

Answer: `??` (null-coalescing) returns the right-hand value if the left is null. `?.` (null-conditional) accesses a member only if the object isn't null, otherwise it returns null.

Q120. What are properties in C#?

Answer: Properties expose fields through get and set accessors, allowing validation and encapsulation. Auto-properties like `public int Id { get; set; }` generate the backing field automatically.

Q121. What are records in C#?

Answer: Records (`C# 9+`) are reference types designed for immutable data. They provide value-based equality, with expressions and concise syntax. They're ideal for DTOs.

Q122. What is the var keyword?

Answer: `var` declares an implicitly typed local variable. The type is inferred at compile time and is still strongly typed.

Q123. What is the difference between var and dynamic?

Answer: `var` is resolved at compile time and is strongly typed. `dynamic` bypasses compile-time type checking and is resolved at run time, which can cause runtime errors.

Q124. What are indexers?

Answer: Indexers let objects be accessed like arrays using `this[int index]`, for example a custom collection class.

Q125. What are partial classes?

Answer: Partial classes split one class definition across multiple files. They're common with generated code.

Q126. What is reflection?

Answer: Reflection inspects types, methods and attributes at run time and can create objects dynamically. It's used by frameworks and serializers, but it's slower than direct calls.

Q127. What are attributes?

Answer: Attributes add metadata to code, such as `[Required]`, `[HttpGet]` or `[Test]`. Frameworks read them to change behavior.

SOLID & Design Patterns

Q128. Explain the SOLID principles.

Answer: S: Single Responsibility (one reason to change). O: Open/Closed (open for extension, closed for modification). L: Liskov Substitution (subtypes must be usable in place of base types). I: Interface Segregation (small, specific interfaces). D: Dependency Inversion (depend on abstractions, not concrete classes).

Q129. Give an example of the Single Responsibility Principle from your banking project.

Answer: Instead of one class handling validation, database access and business logic, I separated them: controllers handle HTTP, services handle business rules like transfer limits, and repositories handle database access. Each class has one reason to change.

Q130. What is Dependency Injection?

Answer: DI is a technique where a class receives its dependencies from outside rather than creating them. It improves testability (you can inject mocks) and loose coupling. ASP.NET Core has a built-in DI container.

Q131. What is the Singleton pattern?

Answer: It ensures only one instance of a class exists, with a global access point. It's implemented with a private constructor and a static instance, made thread-safe with Lazy<T>. In ASP.NET Core, it's usually registered as a singleton service instead.

Q132. What is the Repository pattern?

Answer: It abstracts data access behind an interface like IAccountRepository, so business logic doesn't depend on the database directly. I used it in my banking project with a layered architecture.

Q133. What is the Factory pattern?

Answer: A factory creates objects without exposing the creation logic. The client asks for an object by type or parameter, for example a NotificationFactory that returns email or SMS notifiers.

Q134. What is the Strategy pattern?

Answer: It defines a family of algorithms, puts each in its own class, and makes them interchangeable at run time. An example is different interest calculation strategies for different account types.

Q135. What is the Observer pattern?

Answer: One object (the subject) notifies many observers when its state changes. C# events implement this pattern.

Q136. What is the difference between composition and inheritance?

Answer: Inheritance is an "is-a" relationship. Composition is "has-a", where a class contains other objects. Composition is often preferred because it's more flexible and less tightly coupled.

Q137. What is loose coupling and why is it important?

Answer: Loose coupling means classes depend on abstractions and know little about each other's internals. It makes code easier to test, maintain and change.

.NET Framework & Runtime

Q138. What is .NET?

Answer: .NET is a free, open-source, cross-platform developer platform from Microsoft for building web, cloud, desktop, mobile and AI applications, using languages like C#.

Q139. What is the difference between .NET Framework and .NET Core / .NET 5+?

Answer: .NET Framework is Windows-only and legacy. .NET Core (now simply .NET 5, 6, 7, 8 and later) is cross-platform, open-source, faster, modular and supports containers and microservices.

Q140. What is the CLR?

Answer: The Common Language Runtime is the execution engine of .NET. It handles JIT compilation, memory management (garbage collection), type safety, exception handling and security.

Q141. What is JIT compilation?

Answer: Just-In-Time compilation converts Intermediate Language (IL) into native machine code at run time, when a method is first called.

Q142. What is IL / MSIL?

Answer: Intermediate Language is the CPU-independent code produced when C# is compiled. The CLR JIT-compiles it to native code.

Q143. What are CTS and CLS?

Answer: The Common Type System defines how types are declared and used across .NET languages. The Common Language Specification is a subset of rules that all .NET languages follow, so they can interoperate.

Q144. What is an assembly?

Answer: An assembly is the compiled output (.dll or .exe) containing IL code, metadata and a manifest. It's the unit of deployment and versioning.

Q145. What is NuGet?

Answer: NuGet is the package manager for .NET, used to add and manage third-party libraries, like Newtonsoft.Json or NUnit.

Q146. What is the difference between managed and unmanaged code?

Answer: Managed code runs under the CLR with automatic memory management. Unmanaged code, like C++ or native OS resources, runs outside the CLR and must be cleaned up manually, for example through IDisposable.

ASP.NET Core & Web API

Q147. What is ASP.NET Core?

Answer: ASP.NET Core is a cross-platform, high-performance framework for building web apps, REST APIs and microservices. It has built-in DI, middleware, configuration and logging.

Q148. What is Program.cs in ASP.NET Core?

Answer: It's the application's entry point. It creates the WebApplication builder, registers services in the DI container, configures the middleware pipeline and runs the app. Since .NET 6, it replaced Startup.cs in the minimal hosting model.

Q149. What is middleware?

Answer: Middleware components form a pipeline that handles each HTTP request and response. Each can process the request, pass it to the next one, or short-circuit. Examples include authentication, routing, exception handling and CORS.

Q150. Does the order of middleware matter?

Answer: Yes. Middleware runs in the order it's registered. For example, UseAuthentication must come before UseAuthorization, and exception handling should be registered early to catch errors from later middleware.

Q151. How do you create custom middleware?

Answer: Create a class with a constructor that takes RequestDelegate and an InvokeAsync(HttpContext) method. Do the work, call await _next(context), then register it with app.UseMiddleware<T>(). It's commonly used for logging or global error handling.

Q152. What is dependency injection in ASP.NET Core?

Answer: ASP.NET Core has a built-in DI container. You register services in Program.cs (builder.Services.AddScoped<IAccountService, AccountService>()) and they're injected into controllers through constructors.

Q153. What are service lifetimes: Transient, Scoped and Singleton?

Answer: Transient: a new instance every time it's requested (lightweight, stateless services). Scoped: one instance per HTTP request (DbContext, business services). Singleton: one instance for the app's lifetime (caching, configuration).

Q154. What happens if you inject a scoped service into a singleton?

Answer: It causes a captive dependency: the scoped service lives as long as the singleton, which can lead to bugs like a stale DbContext. ASP.NET Core throws an error in development. Use IServiceScopeFactory to create a scope instead.

Q155. What is a REST API?

Answer: REST is an architectural style where resources are identified by URLs and manipulated with HTTP methods (GET, POST, PUT, PATCH, DELETE). It's stateless and typically uses JSON.

Q156. What are the HTTP methods and their uses?

Answer: GET retrieves data. POST creates. PUT replaces or updates a whole resource. PATCH partially updates. DELETE removes. GET, PUT and DELETE are idempotent; POST is not.

Q157. What does idempotent mean?

Answer: Calling the operation multiple times has the same effect as calling it once. For example, deleting the same account twice leaves the same result.

Q158. What are the common HTTP status codes?

Answer: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Internal Server Error.

Q159. What is the difference between 401 and 403?

Answer: 401 means the user isn't authenticated (not logged in or invalid token). 403 means the user is authenticated but lacks permission, for example a customer trying to access a manager-only endpoint.

Q160. What is the difference between a Controller and ControllerBase?

Answer: ControllerBase provides API features without view support and is used for Web APIs. Controller extends it with view support for MVC.

Q161. What does the [ApiController] attribute do?

Answer: It enables API-specific behaviors: automatic 400 responses for invalid models, binding-source inference ([FromBody], [FromRoute]) and attribute routing requirements.

Q162. What is routing in ASP.NET Core?

Answer: Routing maps incoming URLs to controller actions. Attribute routing uses [Route("api/accounts")] and [HttpGet("{id}")] directly on controllers and actions.

Q163. What is model binding?

Answer: Model binding maps request data (route, query string, body, headers) to action parameters and objects automatically, using sources like [FromBody], [FromQuery] and [FromRoute].

Q164. How do you validate input in a Web API?

Answer: With data annotations like [Required], [Range] and [StringLength] on DTOs. [ApiController] returns 400 automatically when ModelState is invalid. FluentValidation handles more complex rules. In my banking app, I validated amounts and account data before processing.

Q165. What are DTOs and why use them?

Answer: Data Transfer Objects carry data between layers or over the API. They prevent exposing internal entities, control exactly what's sent or received, and improve security.

Q166. What is IActionResult vs ActionResult<T>?

Answer: IActionResult returns any HTTP response. ActionResult<T> also allows returning a typed object, which improves Swagger documentation and readability.

Q167. What are filters in ASP.NET Core?

Answer: Filters run code at specific stages of the request pipeline: Authorization, Resource, Action, Exception and Result filters. They're used for cross-cutting concerns like logging and error handling.

Q168. How do you handle exceptions globally in a Web API?

Answer: With custom exception-handling middleware or app.UseExceptionHandler(). They catch unhandled exceptions, log them, and return a consistent error response (like ProblemDetails) instead of leaking stack traces.

Q169. What is appsettings.json and how do you read configuration?

Answer: appsettings.json stores configuration like connection strings and API settings. You read it via IConfiguration, or bind it to strongly typed classes with the Options pattern (IOptions<T>). Environment-specific files like appsettings.Development.json override values.

Q170. How should you store secrets like API keys?

Answer: Never hardcode them or commit them to Git. Use User Secrets in development, and environment variables or Azure Key Vault in production. That includes Azure OpenAI keys.

Q171. What is logging in ASP.NET Core?

Answer: ASP.NET Core has built-in ILogger<T> with log levels (Trace, Debug, Information, Warning, Error, Critical). It can be extended with providers like Serilog and Application Insights.

Q172. What is Swagger / OpenAPI?

Answer: Swagger generates interactive API documentation from your code, so APIs can be tested in the browser. In ASP.NET Core, it's added with Swashbuckle.

Q173. What is CORS and why is it needed?

Answer: Cross-Origin Resource Sharing lets a browser app from one origin (for example Angular on localhost:4200) call an API on another origin. Without enabling CORS on the API, the browser blocks those requests. I configured it for my Angular + ASP.NET banking app.

Q174. What is authentication vs authorization?

Answer: Authentication verifies who the user is (login). Authorization decides what they can do (permissions).

Q175. What is JWT?

Answer: A JSON Web Token is a compact, signed token with three parts: header, payload (claims) and signature. After login, the server issues a JWT, and the client sends it in the Authorization header as a Bearer token. It's stateless and common for APIs and SPAs like Angular.

Q176. How did you implement role-based access in your banking application?

Answer: Users had roles: Customer and Manager. On login, the role was included in the user's claims, for example in the JWT. API endpoints were protected with [Authorize(Roles = "Manager")] for manager-only actions, and the Angular UI showed menus and routes based on role, using route guards. (Customize with your exact implementation.)

Q177. What is the [Authorize] attribute?

Answer: It restricts access to authenticated users. Adding Roles or Policy parameters restricts it further. [AllowAnonymous] overrides it for public endpoints like login.

Q178. What is policy-based authorization?

Answer: Authorization rules defined as policies with requirements (for example "MinimumAge" or "CanApproveLoans"), registered in Program.cs and applied with [Authorize(Policy = "...")]. It's more flexible than plain roles.

Q179. How do you secure a Web API?

Answer: Use HTTPS, JWT/OAuth authentication, role or policy authorization, input validation, parameterized queries (prevent SQL injection), rate limiting, proper CORS, secret management and no sensitive data in error messages.

Q180. What is API versioning?

Answer: Versioning maintains backward compatibility when APIs change, via the URL (/api/v1/accounts), query string or headers.

Q181. What is caching in ASP.NET Core?

Answer: Caching stores frequently used data to improve performance: in-memory caching (IMemoryCache), distributed caching (Redis) and response caching.

Q182. What is the difference between synchronous and asynchronous controllers?

Answer: Async actions (async Task<IActionResult>) free the thread while waiting for I/O like database or API calls, so the server handles more concurrent requests. Use async for all I/O-bound operations.

Q183. What is Kestrel?

Answer: Kestrel is the default cross-platform web server in ASP.NET Core. In production, it's often placed behind a reverse proxy like IIS, Nginx or a load balancer.

Q184. What is the difference between MVC and Web API?

Answer: MVC returns HTML views for web pages. Web API returns data (JSON or XML) for clients like Angular or mobile apps. In ASP.NET Core both share the same framework.

Q185. What is Minimal API?

Answer: A lightweight way to create APIs in .NET 6+ with very little code, mapping endpoints directly in Program.cs with app.MapGet(). It suits small services and microservices.

Q186. What is the layered architecture you used in your banking project?

Answer: Controllers receive HTTP requests and return responses. Services contain business logic like balance checks and transfer rules. Repositories handle data access to the SQL database. DTOs move data between layers. This separation made the code testable and maintainable.

Entity Framework & Data Access

Q187. What is Entity Framework Core?

Answer: EF Core is Microsoft's ORM (Object-Relational Mapper). It maps C# classes to database tables and lets you query with LINQ instead of raw SQL.

Q188. What are the Code-First and Database-First approaches?

Answer: Code-First: you write C# models and EF creates or updates the database through migrations. Database-First: you generate models from an existing database.

Q189. What are migrations in EF Core?

Answer: Migrations track model changes and apply them to the database schema incrementally, with Add-Migration and Update-Database (or dotnet ef commands).

Q190. What is DbContext?

Answer: DbContext represents a session with the database. It contains DbSet<T> properties for tables, tracks changes and saves them with SaveChanges(). It should be registered as scoped.

Q191. What is lazy, eager and explicit loading?

Answer: Eager loading loads related data up front with Include(). Lazy loading loads related data automatically when it's accessed (can cause the N+1 problem). Explicit loading loads related data manually when needed.

Q192. What is the N+1 query problem?

Answer: One query loads a list, then N extra queries load related data for each item, which hurts performance. Fix it with eager loading (Include) or projections.

Q193. What is AsNoTracking()?

Answer: It disables change tracking for read-only queries, which improves performance and memory use.

Q194. How do you prevent SQL injection in .NET?

Answer: Use parameterized queries, EF Core LINQ, or stored procedures with parameters. Never concatenate user input into SQL strings. Also validate inputs.

Q195. What is ADO.NET?

Answer: ADO.NET is the low-level data access library (SqlConnection, SqlCommand, SqlDataReader). It gives full control over SQL and is faster, but needs more code than EF Core.

Q196. What is Dapper?

Answer: Dapper is a lightweight micro-ORM that maps SQL query results to objects. It's very fast and a middle ground between ADO.NET and EF Core.

Q197. How do you handle transactions in EF Core?

Answer: SaveChanges() is transactional by default. For multiple operations, like a fund transfer that debits one account and credits another, use context.Database.BeginTransaction() and commit only if both succeed, or roll back.

Angular & Front End

Q198. What is Angular?

Answer: Angular is a TypeScript-based front-end framework by Google for building single-page applications (SPAs). It provides components, routing, forms, HTTP client and dependency injection out of the box.

Q199. What is a Single-Page Application?

Answer: An SPA loads one HTML page and updates content dynamically without full page reloads, giving a faster, app-like experience. My banking app's Angular front end was an SPA.

Q200. What are components in Angular?

Answer: Components are the building blocks of the UI. Each has a TypeScript class (logic), an HTML template (view) and CSS (styles), defined with the @Component decorator.

Q201. What are modules in Angular?

Answer: NgModules group related components, directives, pipes and services. Newer Angular versions support standalone components that don't need NgModules.

Q202. What is data binding? What are its types?

Answer: Data binding connects the component and the template. Interpolation {{ value }} and property binding [property] go from component to view. Event binding (event) goes from view to component. Two-way binding [(ngModel)] syncs both ways.

Q203. What are directives?

Answer: Directives change the DOM or behavior. Structural directives (*ngIf, *ngFor, or @if/@for in new syntax) add or remove elements. Attribute directives (ngClass, ngStyle) change appearance. You can also create custom directives.

Q204. What are pipes?

Answer: Pipes transform displayed data in templates, like date, currency, uppercase and async. In the banking dashboard, a currency pipe can format balances. You can create custom pipes too.

Q205. What are services in Angular?

Answer: Services hold reusable logic like API calls or shared data, and are injected into components with DI. They're marked with `@Injectable({ providedIn: 'root' })` to be singletons.

Q206. How do you call a REST API from Angular?

Answer: Use `HttpClient` in a service. For example, `this.http.get<Account[]>('/api/accounts')` returns an `Observable`, which the component subscribes to, or uses with the `async` pipe. That's how my Angular UI integrated with the ASP.NET APIs.

Q207. What are Observables and RxJS?

Answer: RxJS is a library for reactive programming. An `Observable` is a stream of values over time that you subscribe to. `HttpClient` returns `Observables`, and operators like `map`, `filter`, `switchMap` and `catchError` transform them.

Q208. What is the difference between an Observable and a Promise?

Answer: A `Promise` handles a single `async` value and runs immediately. An `Observable` can emit multiple values, is lazy (runs on `subscribe`), can be cancelled (`unsubscribe`), and supports powerful operators.

Q209. What is the async pipe?

Answer: It subscribes to an `Observable` or `Promise` in the template and unsubscribes automatically when the component is destroyed, which prevents memory leaks.

Q210. What are lifecycle hooks in Angular?

Answer: Methods called at specific points in a component's life: `ngOnChanges`, `ngOnInit` (initialization, good for API calls), `ngDoCheck`, `ngAfterViewInit` and `ngOnDestroy` (cleanup, unsubscribing).

Q211. What is the difference between the constructor and ngOnInit?

Answer: The constructor is used for dependency injection and simple setup. `ngOnInit` runs after inputs are set and is where initialization logic like API calls should go.

Q212. How do components communicate?

Answer: Parent to child with `@Input()`. Child to parent with `@Output()` and `EventEmitter`. Between unrelated components through a shared service (often with a `Subject` or `BehaviorSubject`). Newer versions also use signals.

Q213. What is routing in Angular?

Answer: The Angular Router maps URLs to components, enabling navigation in an SPA without reloads. Routes are defined in a `routes` array, and `<router-outlet>` displays the active component.

Q214. What are route guards?

Answer: Guards control navigation: `CanActivate` (can the user enter the route), `CanDeactivate`, `CanMatch` and `resolvers`. I used guards to restrict pages by role, for example manager-only screens.

Q215. What is lazy loading in Angular?

Answer: Loading feature modules or components only when their route is visited, using `loadChildren` or `loadComponent`. It reduces initial bundle size and speeds up startup.

Q216. What are template-driven and reactive forms?

Answer: Template-driven forms are defined in HTML with ngModel and suit simple forms. Reactive forms are defined in TypeScript with FormGroup and FormControl, and are better for complex validation and testability.

Q217. How do you validate forms in Angular?

Answer: Use built-in validators (required, minLength, pattern, min or max) or custom validators, then show error messages based on control state. For example, validate that a transfer amount is positive.

Q218. What are HTTP interceptors?

Answer: Interceptors intercept every HTTP request and response. They're used to attach JWT tokens to headers, handle errors globally, show loaders and log requests.

Q219. How do you attach a JWT token to API requests in Angular?

Answer: With an HTTP interceptor that reads the token from storage and clones the request with an Authorization: Bearer <token> header.

Q220. What is change detection in Angular?

Answer: Angular checks for data changes and updates the DOM. The Default strategy checks the whole tree. OnPush checks only when inputs change or events fire, which improves performance.

Q221. What are Angular signals?

Answer: Signals (Angular 16+) are reactive values that notify consumers when they change, allowing fine-grained, efficient change detection with a simpler reactivity model than RxJS for local state.

Q222. What is TypeScript and why does Angular use it?

Answer: TypeScript is a typed superset of JavaScript. Static typing catches errors at compile time and improves tooling and maintainability, which suits large Angular apps.

Q223. What is the Angular CLI?

Answer: A command-line tool to create projects, generate components and services, and serve, build and test Angular apps: ng new, ng generate component, ng serve, ng build.

Q224. What is AOT compilation?

Answer: Ahead-of-Time compilation compiles templates at build time instead of in the browser, which gives faster rendering, smaller bundles and earlier error detection.

Q225. What is dependency injection in Angular?

Answer: Angular's DI provides instances of services to components and other services through constructors or the inject() function, making code modular and testable.

Q226. What is a BehaviorSubject?

Answer: A Subject that holds a current value and emits it immediately to new subscribers. It's commonly used in services for shared state, like the logged-in user.

Q227. How do you prevent memory leaks in Angular?

Answer: Unsubscribe from Observables in ngOnDestroy, use the async pipe, use takeUntil or takeUntilDestroyed, and avoid keeping references to destroyed components.

Q228. How did you build the dashboard in your banking app?

Answer: The dashboard component called Angular services that fetched account balances and recent transactions from the ASP.NET APIs through HttpClient. Data was displayed with *ngFor tables and formatted with the currency and date pipes, and the view adapted by user role. (Customize to your implementation.)

Q229. What is the difference between HTML and the DOM?

Answer: HTML is the markup text. The DOM is the browser's in-memory tree representation of the page, which JavaScript and Angular manipulate.

Q230. What is responsive design?

Answer: Designing UIs that adapt to different screen sizes using flexible layouts, CSS media queries, Flexbox or Grid, and relative units.

Q231. What is the difference between Flexbox and CSS Grid?

Answer: Flexbox is one-dimensional (rows or columns) and good for components. Grid is two-dimensional (rows and columns) and good for page layouts.

Q232. What is the difference between localStorage, sessionStorage and cookies?

Answer: localStorage persists until cleared. sessionStorage lasts only for the tab session. Cookies are sent with every HTTP request and can be HttpOnly, which is safer for auth tokens against XSS.

SQL & Databases

Q233. What is a DBMS and an RDBMS?

Answer: A DBMS is software for storing and managing data. An RDBMS stores data in related tables with rows and columns and uses SQL. Examples are MySQL, SQL Server and PostgreSQL.

Q234. What is a primary key?

Answer: A column or set of columns that uniquely identifies each row. It can't be NULL or duplicated, like AccountId.

Q235. What is a foreign key?

Answer: A column that references another table's primary key, enforcing referential integrity. For example, Transactions.AccountId references Accounts.AccountId.

Q236. What is the difference between a primary key and a unique key?

Answer: Both enforce uniqueness. A table has only one primary key, which can't be NULL. It can have many unique keys, which usually allow NULL (one NULL in SQL Server; multiple in MySQL).

Q237. What is normalization?

Answer: Organizing tables to reduce redundancy and improve integrity. 1NF: atomic values. 2NF: no partial dependency on part of a composite key. 3NF: no transitive dependency (non-key columns depend only on the key). BCNF is a stricter 3NF.

Q238. What is denormalization?

Answer: Intentionally adding redundancy to speed up reads, for example in reporting or analytics, at the cost of more storage and harder updates.

Q239. What are the types of JOINS?

Answer: INNER JOIN returns matching rows in both tables. LEFT JOIN returns all left rows plus matches. RIGHT JOIN returns all right rows plus matches. FULL OUTER JOIN returns all rows from both. CROSS JOIN returns the Cartesian product. A SELF JOIN joins a table to itself.

Q240. What is the difference between WHERE and HAVING?

Answer: WHERE filters rows before grouping. HAVING filters groups after GROUP BY and can use aggregate functions, like HAVING COUNT(*) > 5.

Q241. What is GROUP BY?

Answer: It groups rows with the same values so aggregate functions (COUNT, SUM, AVG, MIN, MAX) can be applied per group. For example, total transaction amount per account.

Q242. What is the difference between DELETE, TRUNCATE and DROP?

Answer: DELETE removes selected rows, can use WHERE, is logged and can be rolled back. TRUNCATE removes all rows quickly with minimal logging and resets identity. DROP removes the entire table structure.

Q243. What is the difference between UNION and UNION ALL?

Answer: UNION combines results and removes duplicates. UNION ALL keeps duplicates and is faster.

Q244. What is an index?

Answer: A data structure (usually a B-tree) that speeds up data retrieval on columns used in WHERE, JOIN and ORDER BY. The trade-off is slower writes and extra storage.

Q245. What is the difference between clustered and non-clustered indexes?

Answer: A clustered index defines the physical order of data, so there's only one per table (usually the primary key). A non-clustered index is a separate structure with pointers to the rows, and a table can have many.

Q246. What are ACID properties?

Answer: Atomicity (all or nothing), Consistency (valid state before and after), Isolation (transactions don't interfere) and Durability (committed data persists). These are critical for banking transactions.

Q247. Explain a transaction using a fund transfer example.

Answer: BEGIN TRANSACTION, debit account A, credit account B, then COMMIT if both succeed. If either fails, ROLLBACK so money is never lost or duplicated. That's atomicity.

Q248. What are transaction isolation levels?

Answer: Read Uncommitted (dirty reads possible), Read Committed (the default in SQL Server; prevents dirty reads), Repeatable Read (prevents non-repeatable reads) and Serializable (full isolation, prevents phantoms). Higher isolation means lower concurrency.

Q249. What are dirty reads, non-repeatable reads and phantom reads?

Answer: Dirty read: reading uncommitted data. Non-repeatable read: the same row returns different values within one transaction. Phantom read: new rows appear when a query is re-run.

Q250. What is a stored procedure?

Answer: Precompiled SQL code stored in the database that can take parameters. It improves performance, security and reusability.

Q251. What is the difference between a stored procedure and a function?

Answer: A function must return a value, can be used in SELECT, and can't usually modify data. A stored procedure may return values or result sets, can modify data and handle transactions, but can't be used inside a SELECT.

Q252. What is a view?

Answer: A virtual table based on a SELECT query. It simplifies complex queries and restricts access to specific columns or rows.

Q253. What is a trigger?

Answer: SQL code that runs automatically on INSERT, UPDATE or DELETE. For example, writing an audit log entry whenever an account balance changes.

Q254. What is a subquery? What is a correlated subquery?

Answer: A subquery is a query nested inside another. A correlated subquery references the outer query and runs once per outer row, which can be slow.

Q255. What is a CTE?

Answer: A Common Table Expression (WITH ... AS) is a named temporary result set that makes complex queries readable. It supports recursion, for example for hierarchies.

Q256. What are window functions?

Answer: Functions that compute across a set of rows related to the current row without collapsing them: ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD() and SUM() OVER (PARTITION BY ...).

Q257. Write a query to find the second-highest salary.

Answer: SELECT MAX(Salary) FROM Employees WHERE Salary < (SELECT MAX(Salary) FROM Employees); Or use DENSE_RANK() in a CTE and select the row where rank = 2.

Q258. Write a query to find the Nth highest salary.

Answer: WITH R AS (SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rn FROM Employees) SELECT DISTINCT Salary FROM R WHERE rn = N;

Q259. Write a query to find duplicate records.

Answer: SELECT Email, COUNT(*) FROM Users GROUP BY Email HAVING COUNT(*) > 1;

Q260. How do you delete duplicate rows but keep one?

Answer: WITH C AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY Email ORDER BY Id) AS rn FROM Users) DELETE FROM C WHERE rn > 1; (SQL Server syntax.)

Q261. Write a query to get each account's total transaction amount.

Answer: SELECT AccountId, SUM(Amount) AS Total FROM Transactions GROUP BY AccountId;

Q262. Write a query to find customers who have no transactions.

Answer: SELECT c.* FROM Customers c LEFT JOIN Transactions t ON c.Id = t.CustomerId WHERE t.Id IS NULL;

Q263. Write a query to get the last 5 transactions of an account.

Answer: SELECT * FROM Transactions WHERE AccountId = @id ORDER BY TransactionDate DESC LIMIT 5; (MySQL.) In SQL Server use SELECT TOP 5 ... ORDER BY TransactionDate DESC.

Q264. What is the difference between CHAR and VARCHAR?

Answer: CHAR is fixed-length and padded with spaces. VARCHAR is variable-length and stores only the actual characters, so it's more space-efficient for varying data.

Q265. What are constraints in SQL?

Answer: Rules on columns: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK (for example Balance >= 0) and DEFAULT.

Q266. What is SQL injection and how do you prevent it?

Answer: An attack where malicious SQL is inserted through user input to manipulate queries. Prevent it with parameterized queries or prepared statements, ORMs, input validation and least-privilege database accounts.

Q267. How do you optimize a slow SQL query?

Answer: Check the execution plan, add proper indexes, avoid SELECT *, filter early, avoid functions on indexed columns in WHERE, replace correlated subqueries with joins, and paginate large results.

Q268. What is an execution plan?

Answer: A roadmap showing how the database executes a query: scans, seeks, joins and costs. It's used to find bottlenecks like table scans.

Q269. What is the difference between an index scan and an index seek?

Answer: A seek navigates directly to matching rows using the index, which is fast. A scan reads the whole index or table, which is slower.

Q270. What is the difference between SQL and NoSQL databases?

Answer: SQL databases are relational, have a fixed schema, ACID and joins, like MySQL. NoSQL databases are non-relational with flexible schemas and scale horizontally: document stores (MongoDB), key-value (Redis), column and graph databases.

Q271. What is the difference between MySQL and SQL Server?

Answer: Both are RDBMSs. MySQL is open-source and popular for web apps. SQL Server is Microsoft's RDBMS, closely integrated with .NET and Azure. Syntax differs slightly (LIMIT vs TOP, AUTO_INCREMENT vs IDENTITY).

Q272. What is database sharding and replication?

Answer: Sharding splits data across multiple databases for horizontal scaling. Replication copies data to multiple servers for availability and read scaling.

Q273. What is a deadlock in a database?

Answer: Two transactions each hold a lock the other needs, so both wait forever. The database kills one (the victim). Reduce deadlocks by accessing tables in a consistent order and keeping transactions short.

Q274. What is the COALESCE function?

Answer: It returns the first non-NULL value in a list, for example COALESCE(Phone, 'N/A').

Q275. What is the difference between COUNT(*) and COUNT(column)?

Answer: COUNT(*) counts all rows. COUNT(column) counts only rows where that column isn't NULL.

Q276. What is an ER diagram?

Answer: An Entity-Relationship diagram visually represents entities (tables), attributes and relationships (one-to-one, one-to-many, many-to-many). It's used when designing databases, like Customer, Account and Transaction in a banking system.

Q277. How would you design tables for a banking application?

Answer: Users (Id, Name, Email, PasswordHash, Role), Accounts (Id, UserId FK, AccountNumber, Type, Balance), Transactions (Id, AccountId FK, Type, Amount, Date, Status). Add indexes on foreign keys and dates, a CHECK constraint so the balance can't go negative, and transactions for transfers.

Generative AI, Azure OpenAI & Prompt Engineering

Q278. What is Generative AI?

Answer: Generative AI refers to models that create new content (text, code, images, audio) by learning patterns from large datasets. Examples are GPT models for text and DALL·E for images.

Q279. What is a Large Language Model?

Answer: An LLM is a deep learning model, based on the Transformer architecture, trained on huge text data to predict the next token. That lets it understand and generate human-like text, answer questions, summarize and write code.

Q280. What is the Transformer architecture?

Answer: A neural network architecture introduced in "Attention Is All You Need" (2017). It uses self-attention to weigh the importance of every token relative to others, processes sequences in parallel, and is the foundation of GPT and BERT.

Q281. What is self-attention?

Answer: A mechanism where each token computes how much attention to pay to every other token in the sequence using Query, Key and Value vectors. It lets the model capture context and long-range relationships.

Q282. What is a token?

Answer: A token is a chunk of text (a word, part of a word or punctuation) that LLMs process. Pricing, context limits and max output are measured in tokens. Roughly, 1 token is about 4 English characters.

Q283. What is a context window?

Answer: The maximum number of tokens (input plus output) a model can handle in one request. Longer documents must be chunked, summarized or retrieved selectively (RAG).

Q284. What is Azure OpenAI Service?

Answer: Microsoft's managed service that provides OpenAI models (like GPT-4o and embedding models) through Azure, with enterprise security, private networking, data privacy, content filtering and regional compliance.

Q285. What is the difference between OpenAI's API and Azure OpenAI?

Answer: They offer the same model families. Azure OpenAI adds enterprise features: Azure AD authentication, VNet and private endpoints, compliance certifications, regional data residency, built-in content filtering and integration with Azure services. It uses deployment names instead of direct model names.

Q286. How do you call Azure OpenAI from a .NET application?

Answer: Use the Azure.AI.OpenAI SDK (or REST). Create a client with the endpoint and key (or managed identity), then call the chat completions API with a deployment name and a list of messages (system, user, assistant), plus parameters like temperature and max tokens. Store keys in Key Vault, not in code.

Q287. What are system, user and assistant messages?

Answer: The system message sets the model's behavior, role and rules. User messages are the inputs. Assistant messages are the model's previous replies, included to keep conversation history.

Q288. What is prompt engineering?

Answer: Designing and refining inputs to get accurate, relevant and consistent outputs from an LLM. It uses clear instructions, context, examples, output formats and constraints.

Q289. How did you improve GenAI accuracy by 18% and reduce errors by 12%?

Answer: Through iterative prompt refinement. I made instructions clearer and more specific, added relevant context and few-shot examples, defined strict output formats, added constraints to reduce hallucinations, and tuned parameters like temperature. I tested each version against a fixed set of test cases, measured correct vs incorrect responses, and kept the changes that improved results. (Customize with your exact measurement method.)

Q290. How do you measure the accuracy of an LLM's responses?

Answer: Create a test set of inputs with expected outputs. Run prompts against it and score responses by exact match, rubric-based human review, or automated checks like format validation, keyword presence or an LLM-as-judge. Compare accuracy and error rates across prompt versions.

Q291. What is zero-shot, one-shot and few-shot prompting?

Answer: Zero-shot: only instructions, no examples. One-shot: one example. Few-shot: several input-output examples in the prompt, which guide the model to the desired format and reasoning.

Q292. What is chain-of-thought prompting?

Answer: Asking the model to reason step by step before answering. It improves accuracy on complex reasoning tasks like math and logic.

Q293. What is temperature in LLMs?

Answer: It controls randomness. Low temperature (0 to 0.3) gives focused, deterministic output, which is good for factual or structured tasks. High temperature (0.7 to 1+) gives more creative, varied output.

Q294. What is top_p?

Answer: Nucleus sampling: the model picks from the smallest set of tokens whose cumulative probability reaches p. It's another way to control randomness. Usually you tune either temperature or top_p, not both.

Q295. What is a hallucination in LLMs?

Answer: When the model generates confident but false or made-up information.

Q296. How do you reduce hallucinations?

Answer: Use RAG to ground answers in real data, instruct the model to answer only from the provided context and say "I don't know" otherwise, lower the temperature, ask for citations, use structured outputs and validate responses.

Q297. What is RAG?

Answer: Retrieval-Augmented Generation. Relevant documents are retrieved from a knowledge base (usually via vector search) and added to the prompt, so the LLM answers using up-to-date, domain-specific data without retraining.

Q298. Explain the RAG pipeline step by step.

Answer: (1) Ingest documents and split them into chunks. (2) Convert chunks into embeddings. (3) Store them in a vector database. (4) At query time, embed the user question. (5) Retrieve the most similar chunks. (6) Add them to the prompt as context. (7) The LLM generates a grounded answer, optionally with citations.

Q299. What are embeddings?

Answer: Numerical vector representations of text that capture meaning. Similar texts have vectors close together, which enables semantic search. An example is Azure OpenAI's text-embedding models.

Q300. What is a vector database?

Answer: A database optimized for storing embeddings and doing fast similarity search. Examples: Azure AI Search, Pinecone, FAISS, Chroma, pgvector.

Q301. What is cosine similarity?

Answer: A measure of similarity between two vectors based on the angle between them (1 = identical direction). It's commonly used to compare embeddings.

Q302. What is chunking and why does chunk size matter?

Answer: Splitting documents into smaller pieces for embedding. Chunks that are too big add noise and cost tokens; chunks that are too small lose context. Overlapping chunks help preserve context across boundaries.

Q303. What is Azure AI Search?

Answer: Azure's search service that supports keyword, vector and hybrid search with semantic ranking. It's commonly used as the retrieval layer in RAG solutions with Azure OpenAI.

Q304. What is fine-tuning? When would you use it over RAG?

Answer: Fine-tuning further trains a model on your own examples to change its style, format or behavior. Use RAG for changing or factual knowledge. Use fine-tuning for consistent tone or format, specialized tasks, or shorter prompts. They can be combined.

Q305. What is Agentic AI?

Answer: AI systems where an LLM acts as an agent. It plans multi-step tasks, decides which tools or APIs to call, observes results and iterates toward a goal with some autonomy, instead of answering a single prompt.

Q306. What is function calling or tool calling?

Answer: The model is given tool definitions (name, description, JSON parameters) and returns a structured request to call a function. The application executes the function and sends the result back to the model. This is the foundation of agents.

Q307. What frameworks are used for building agents?

Answer: Semantic Kernel (Microsoft, good for .NET), LangChain, LangGraph, AutoGen and the Azure AI Agent Service. They manage prompts, memory, tools and orchestration.

Q308. What is Semantic Kernel?

Answer: Microsoft's open-source SDK for integrating LLMs into C#, Python and Java apps. It supports plugins (functions), planners, memory and connectors to Azure OpenAI, and is a natural fit for .NET developers.

Q309. How did you build a chatbot with Azure OpenAI?

Answer: The backend received user messages, kept recent conversation history, added a system prompt defining the bot's role and rules, and called the Azure OpenAI chat completions API. The response was returned to the UI. I handled errors and timeouts, limited history to stay within token limits, and applied content filtering. (Customize to your project.)

Q310. How do you maintain conversation memory in a chatbot?

Answer: Send previous messages with each request. For long conversations, keep only the last N turns, summarize older turns, or store history in a database and retrieve relevant parts.

Q311. How do you get structured JSON output from an LLM?

Answer: Specify the exact JSON schema in the prompt, use JSON mode or structured outputs if the model supports it, use a low temperature, then validate and parse the response in code, with retries on failure.

Q312. What are content filters in Azure OpenAI?

Answer: Built-in safety systems that detect and block harmful content (hate, sexual, violence, self-harm) in both prompts and responses, with configurable severity levels.

Q313. What is prompt injection and how do you defend against it?

Answer: An attack where user input, or retrieved content, contains instructions that override the system prompt. Defend by separating instructions from data, validating and sanitizing inputs, limiting tool permissions, filtering outputs and never trusting model output for sensitive actions without checks.

Q314. How do you handle rate limits (429 errors) in Azure OpenAI?

Answer: Implement retries with exponential backoff (for example with Polly in .NET), respect the retry-after header, queue requests, reduce token usage and request higher quota or add deployments if needed.

Q315. How do you reduce the cost of LLM usage?

Answer: Use shorter, focused prompts, cache repeated responses, choose smaller models for simple tasks, limit max tokens, trim conversation history and use RAG to send only relevant context.

Q316. What is streaming in LLM responses?

Answer: Returning tokens as they're generated (server-sent events) instead of waiting for the full response. It improves perceived speed in chat UIs.

Q317. What is the difference between GPT-4o and GPT-4o-mini style models?

Answer: Larger models are more capable at complex reasoning but slower and costlier. Smaller "mini" models are faster and cheaper, which suits simple, high-volume tasks. Choose based on the accuracy vs cost trade-off.

Q318. What is Amazon Q?

Answer: Amazon Q is AWS's generative AI assistant. Amazon Q Developer helps developers write, explain, debug and document code, and answer AWS questions. I used it to speed up coding and debugging in my banking project.

Q319. How do you use AI coding assistants responsibly?

Answer: Review all generated code, understand it before committing, test it, check for security issues and licensing concerns, and never paste confidential data into unapproved tools.

Q320. What is responsible AI?

Answer: Building AI that is fair, reliable, safe, private, secure, transparent and accountable. Microsoft's Responsible AI principles cover these areas.

Q321. What is the difference between discriminative and generative models?

Answer: Discriminative models learn boundaries to classify data (like a CNN classifying gestures). Generative models learn the data distribution and can generate new samples (like GPT).

Q322. What is NLP?

Answer: Natural Language Processing is the AI field that enables machines to understand and generate human language: tokenization, sentiment analysis, named entity recognition, translation, summarization.

Q323. What is the difference between BERT and GPT?

Answer: BERT is an encoder-only, bidirectional model, good for understanding tasks like classification. GPT is a decoder-only, autoregressive model, good for generating text.

Q324. What is a system prompt, and what makes a good one?

Answer: The instruction that defines the model's role, scope, tone, rules and output format. A good one is clear, specific, includes constraints (what not to do), handles edge cases, and gives examples where needed.

Q325. What is grounding?

Answer: Tying the model's output to specific, trusted data sources (as in RAG), so answers are factual and verifiable.

Q326. How do you evaluate a RAG system?

Answer: Evaluate retrieval (are the right chunks retrieved: precision and recall) and generation (groundedness, relevance, correctness, completeness). Azure AI Foundry evaluation tools and frameworks like RAGAS help.

Q327. What is Azure AI Foundry (Azure AI Studio)?

Answer: Azure's platform for building, testing, evaluating and deploying GenAI apps and agents. It gives access to models, prompt flow, evaluation and deployment tools in one place.

Q328. What is prompt versioning, and why is it useful?

Answer: Tracking changes to prompts like code, with versions and test results. It lets you compare performance, roll back bad changes and show improvements, like my 18% accuracy gain.

Automation, Power Automate & My Automation Tools

Q329. What is Microsoft Power Automate?

Answer: A low-code Microsoft service for automating workflows across apps and services like Outlook, SharePoint, Excel, Teams and 1,000+ connectors, using triggers, actions and conditions.

Q330. What are the types of flows in Power Automate?

Answer: Cloud flows: automated (event-triggered), instant (button-triggered) and scheduled. Desktop flows (RPA) automate desktop and legacy UI tasks. Business process flows guide users through stages.

Q331. What is a trigger and an action in Power Automate?

Answer: A trigger starts the flow, like "When a new email arrives (V3)". Actions are the steps that follow, like parsing the email, adding a row to Excel or sending a Teams message.

Q332. How did your Outlook email automation work?

Answer: An automated cloud flow triggered on new emails in Outlook, filtered by conditions like sender, subject or folder. It extracted the key details from the email subject and body using expressions, parsing and conditions, and saved them as structured rows in Excel, SharePoint or a List, so the data could be tracked and analyzed without manual entry. (Customize with your actual trigger, filters and destination.)

Q333. How do you extract data from an email body in Power Automate?

Answer: Use expressions like split(), substring(), indexOf() and trim(), use "Html to text" to clean HTML bodies, parse JSON if structured, or use AI Builder models for unstructured text.

Q334. What are expressions in Power Automate?

Answer: Functions written in Workflow Definition Language to manipulate data, like concat(), formatDateTime(), utcNow(), split(), if(), length() and coalesce().

Q335. How do you handle errors in Power Automate?

Answer: Use "Configure run after" (run on failure or timeout), Scope actions as try/catch blocks, retry policies and failure notifications. Check the run history for debugging.

Q336. What is a Scope in Power Automate?

Answer: A container that groups actions. Combined with run-after settings, scopes implement try/catch/finally error handling.

Q337. What is "Apply to each"?

Answer: A loop action that processes each item in an array, like each attachment or each row. It can run in parallel with concurrency control.

Q338. What are the limitations of Power Automate?

Answer: API throttling and request limits by license, execution time limits, premium connector costs, complex logic becoming hard to maintain, and limited debugging compared to code.

Q339. When would you choose Power Automate over writing code?

Answer: For workflow automation across Microsoft 365 apps and standard connectors, where speed and low maintenance matter. For complex logic, high performance or heavy processing, custom code is better.

Q340. What is RPA?

Answer: Robotic Process Automation uses software bots to mimic human actions on UIs, like clicking, typing and copying data, to automate repetitive rule-based tasks. Power Automate Desktop is Microsoft's RPA tool.

Q341. What is the PTA automation tool you built?

Answer: *(Explain in your own words.)* It automates a repetitive, rule-based process that my team did manually. It takes the required inputs, processes them automatically, and runs built-in validation checks to ensure the output is accurate. It reduced a 1.25-hour manual task to about 1 minute, a 98% reduction in manual effort.

Q342. How did you calculate the 98% effort reduction?

Answer: The manual process took about 75 minutes and the tool takes about 1 minute. $(75 - 1) / 75 \times 100 \approx 98.7\%$, which I rounded down to 98%.

Q343. What validations did you build into the PTA tool?

Answer: *(Customize.)* Examples: checking required fields are present, validating data formats and ranges, cross-checking values against source data, detecting duplicates, and flagging mismatches in a report instead of silently continuing.

Q344. How did you gather requirements for your automation tools?

Answer: I observed teammates doing the task, documented each step, identified inputs, outputs, rules and exceptions, confirmed them with the team, and prioritized the steps that consumed the most time or caused the most errors.

Q345. How did you test your automation tools?

Answer: I compared tool output with manually produced output on real sample data, tested edge cases like missing or invalid data, fixed issues, and ran a pilot with the team before full rollout.

Q346. What is the Audit Log Validator?

Answer: *(Explain in your own words.)* A tool that automatically reads audit logs and checks them against expected rules, like required entries, correct formats, and consistency with the work done. It verifies the whole team's work within minutes and highlights discrepancies, replacing slow manual review.

Q347. What is an audit log and why is it important?

Answer: A chronological record of actions in a system (who did what, when). It's important for accountability, security, compliance, debugging and detecting unauthorized changes.

Q348. How would you design a log validation tool?

Answer: Parse logs into structured records, define validation rules (required fields, formats, sequences, allowed values, cross-references), run each record through the rules, collect failures with clear reasons, and output a summary report. Make rules configurable so new checks are easy to add.

Q349. How do you parse log files efficiently?

Answer: Read line by line or stream instead of loading everything into memory, use regular expressions or structured parsing (JSON, CSV), handle malformed lines gracefully, and use async I/O for large files.

Q350. What are regular expressions and where did you use them?

Answer: Regex is a pattern-matching language for text. It's useful for validating formats (dates, IDs, emails) and extracting values from logs or emails.

Q351. How do you make an automation tool maintainable?

Answer: Keep rules configurable, write modular code, log each run, handle errors clearly, document usage, add tests, and version the tool so updates are safe.

Q352. How do you ensure automation doesn't introduce errors?

Answer: Build validation into every step, compare outputs with known correct results, flag anything uncertain for human review, log all actions and roll out gradually.

Q353. How did you drive adoption of your tools within the team?

Answer: I demonstrated time saved with real examples, wrote simple usage instructions, supported teammates during the first runs, and improved the tools based on their feedback.

Q354. What would you improve in your automation tools if you had more time?

Answer: (*Customize.*) Examples: a simple UI, scheduling so it runs automatically, better reporting dashboards, more configurable validation rules, and automated tests.

Q355. How do you identify which processes to automate?

Answer: Look for tasks that are repetitive, rule-based, frequent, time-consuming and error-prone. Estimate time saved against build effort and start with the highest return.

Q356. What is the difference between automation and AI?

Answer: Automation follows predefined rules to execute tasks. AI can learn patterns and handle unstructured or uncertain inputs. Combining them, for example using an LLM to extract data from unstructured emails, creates intelligent automation.

Q357. How could GenAI improve your Outlook automation?

Answer: An LLM could extract details from unstructured email text more reliably than fixed parsing rules, classify emails by intent, summarize threads and draft replies, with outputs validated before saving.

Q358. What tools or languages can be used to build automation tools?

Answer: Python (pandas, openpyxl), C#/.NET console or desktop apps, PowerShell scripts, Excel VBA and Power Automate cloud or desktop flows, depending on the systems involved. (Mention the ones you actually used.)

Q359. How do you handle Excel files programmatically?

Answer: In Python with pandas or openpyxl. In C# with EPPlus, ClosedXML or the Open XML SDK. In Power Automate with the Excel Online connector (data must be in a table).

Q360. What is a scheduled job, and how would you schedule an automation?

Answer: A task that runs automatically at set times. Use Power Automate scheduled flows, Windows Task Scheduler, cron jobs, Azure Functions timer triggers or Hangfire in .NET.

Unit Testing, NUnit & Code Quality

Q361. What is unit testing?

Answer: Testing the smallest units of code (methods or classes) in isolation to verify they behave correctly. It catches bugs early, supports refactoring and documents expected behavior.

Q362. What is NUnit?

Answer: A popular open-source unit testing framework for .NET. Tests are written with attributes like [TestFixture], [Test], [SetUp] and [TestCase], and verified with assertions.

Q363. What are the common NUnit attributes?

Answer: [TestFixture] marks a test class. [Test] marks a test method. [SetUp]/[TearDown] run before/after each test. [OneTimeSetUp]/[OneTimeTearDown] run once per fixture. [TestCase] supplies parameterized inputs. [Ignore] skips a test.

Q364. What is the AAA pattern?

Answer: Arrange (set up objects and data), Act (call the method under test), Assert (verify the result). It keeps tests clear and consistent.

Q365. Write a sample NUnit test for a withdrawal method.

Answer: [Test] public void Withdraw_AmountGreaterThanBalance_Throws() { var acc = new Account(100); Assert.Throws<InsufficientBalanceException>(() => acc.Withdraw(200)); } It verifies that business rules are enforced.

Q366. What is the difference between NUnit, xUnit and MSTest?

Answer: All three are .NET test frameworks. NUnit uses [Test]/[TestCase] and has rich assertions. xUnit uses [Fact]/[Theory], creates a new instance per test and is the modern default for .NET Core. MSTest is Microsoft's framework with [TestMethod].

Q367. What is mocking? Which library have you used?

Answer: Mocking replaces real dependencies (database, APIs) with fake objects so a unit is tested in isolation. Moq is the common .NET library: `mock.Setup(x => x.GetBalance(1)).Returns(500);`

Q368. What is the difference between a mock, stub and fake?

Answer: A stub returns predefined data. A mock also verifies interactions (was a method called, and how). A fake is a lightweight working implementation, like an in-memory database.

Q369. What is code coverage?

Answer: The percentage of code executed by tests. It helps find untested code, but high coverage doesn't guarantee good tests. Focus on meaningful business logic and edge cases.

Q370. What is the difference between unit, integration and end-to-end testing?

Answer: Unit tests check isolated components. Integration tests check how components work together (API plus database). End-to-end tests check the full user flow through the UI.

Q371. What is TDD?

Answer: Test-Driven Development: write a failing test first (Red), write the minimum code to pass (Green), then improve the code (Refactor).

Q372. What is a parameterized test?

Answer: One test method run with multiple inputs, using [TestCase(100, 50, 50)] in NUnit. It reduces duplication when testing many scenarios.

Q373. How do you test code that depends on a database?

Answer: For unit tests, mock the repository interface. For integration tests, use an in-memory or SQLite provider, or a test database or container, and reset data between tests.

Q374. What makes a good unit test?

Answer: It's fast, isolated, repeatable, independent of other tests, tests one behavior, has a clear name (Method_Scenario_ExpectedResult) and meaningful assertions.

Q375. What is regression testing?

Answer: Re-running existing tests after changes to make sure nothing that previously worked is broken. Automated unit tests in CI pipelines make this easy.

Q376. What do you look for in a code review?

Answer: Correctness, readability, naming, adherence to standards, error handling, security issues, performance, duplication, test coverage and whether the change solves the requirement.

Q377. How do you give constructive feedback in code reviews?

Answer: Be specific and respectful, explain the reason, suggest alternatives, separate must-fix issues from optional suggestions, and appreciate good work.

Q378. What is clean code?

Answer: Code that is easy to read and maintain: meaningful names, small focused methods, no duplication (DRY), clear structure, proper error handling and comments only where needed.

Q379. What are DRY, KISS and YAGNI?

Answer: DRY: Don't Repeat Yourself. KISS: Keep It Simple. YAGNI: You Aren't Gonna Need It (don't build features before they're needed).

Q380. What is refactoring?

Answer: Improving the internal structure of code without changing its behavior, like renaming, extracting methods or removing duplication. Tests make refactoring safe.

DevOps, Git, CI/CD & Cloud

Q381. What is DevOps?

Answer: A culture and set of practices that combine development and operations to deliver software faster and more reliably, through automation, CI/CD, monitoring and collaboration.

Q382. What is Git?

Answer: A distributed version control system that tracks code changes, supports branching and merging, and lets teams collaborate. Every developer has the full history locally.

Q383. What is the difference between Git and GitHub?

Answer: Git is the version control tool. GitHub is a cloud platform for hosting Git repositories, with pull requests, issues, Actions (CI/CD) and collaboration features.

Q384. What are the common Git commands?

Answer: git clone, git status, git add, git commit -m, git push, git pull, git fetch, git branch, git checkout / git switch, git merge, git rebase, git log, git stash, git reset and git revert.

Q385. What is the difference between git fetch and git pull?

Answer: fetch downloads remote changes without merging. pull is fetch plus merge (or rebase) into your current branch.

Q386. What is the difference between merge and rebase?

Answer: Merge combines branches with a merge commit and preserves history. Rebase replays your commits on top of another branch, giving a linear history. Don't rebase shared public branches.

Q387. How do you resolve a merge conflict?

Answer: Open the conflicting files, review the conflict markers (<<<<<<, =====, >>>>>>), keep the correct changes, remove the markers, test, then git add and commit.

Q388. What is the difference between git reset and git revert?

Answer: reset moves the branch pointer back and can discard commits, so it rewrites history (avoid on shared branches). revert creates a new commit that undoes a previous one, which is safe for shared branches.

Q389. What is git stash?

Answer: It temporarily saves uncommitted changes so you can switch branches, then restore them with git stash pop.

Q390. What is a pull request?

Answer: A request to merge changes from one branch into another. It enables code review, discussion and automated checks before merging.

Q391. What branching strategy have you used?

Answer: Feature branching: create a branch per feature or bug from main or develop, raise a PR, get it reviewed, pass the CI checks, then merge. GitFlow and trunk-based development are other common strategies.

Q392. What is .gitignore?

Answer: A file listing files and folders Git should ignore, like bin/, obj/, node_modules/ and secrets or config files.

Q393. What is CI/CD?

Answer: Continuous Integration: automatically build and test code on every commit. Continuous Delivery/Deployment: automatically release tested code to environments. Together they make releases fast and reliable.

Q394. What is Jenkins?

Answer: An open-source automation server for building CI/CD pipelines. Pipelines are defined in a Jenkinsfile with stages like Build, Test and Deploy, and there's a large plugin ecosystem.

Q395. What is a Jenkinsfile?

Answer: A text file stored in the repository that defines a Jenkins pipeline as code, either declarative or scripted. Example stages: checkout, restore, build, test, publish, deploy.

Q396. Describe a CI/CD pipeline for a .NET application.

Answer: On a push or PR: checkout code, dotnet restore, dotnet build, dotnet test (NUnit), code quality or security scans, dotnet publish, build a Docker image, push it to a registry, deploy to staging, run smoke tests, then deploy to production with approval.

Q397. What are GitHub Actions and Azure DevOps Pipelines?

Answer: Both are CI/CD services. GitHub Actions uses YAML workflows in .github/workflows. Azure DevOps Pipelines uses YAML pipelines and integrates with Azure Boards, Repos and Artifacts.

Q398. What is Docker?

Answer: A platform for packaging an application and its dependencies into lightweight, portable containers that run consistently in any environment, which solves "it works on my machine."

Q399. What is the difference between a container and a virtual machine?

Answer: A container shares the host OS kernel, so it's lightweight and starts in seconds. A VM includes a full guest OS, so it's heavier but has stronger isolation.

Q400. What is a Dockerfile?

Answer: A script with instructions to build a Docker image: FROM (base image), WORKDIR, COPY, RUN, EXPOSE and ENTRYPOINT/CMD.

Q401. What is a multi-stage Docker build for .NET?

Answer: One stage uses the SDK image to restore, build and publish. The final stage copies only the published output into the smaller ASP.NET runtime image. This gives smaller, more secure images.

Q402. What is the difference between a Docker image and a container?

Answer: An image is a read-only template (a blueprint). A container is a running instance of an image.

Q403. What is Docker Compose?

Answer: A tool to define and run multi-container apps (API plus database, for example) with a docker-compose.yml file, started with docker compose up.

Q404. What are the common Docker commands?

Answer: docker build -t name ., docker run -p 8080:80 name, docker ps, docker images, docker stop, docker logs, docker exec -it, docker pull and docker push.

Q405. What is Kubernetes?

Answer: An open-source container orchestration platform that automates deployment, scaling, self-healing, load balancing and rolling updates of containerized applications.

Q406. What are Pods, Deployments and Services in Kubernetes?

Answer: A Pod is the smallest deployable unit (one or more containers). A Deployment manages replicas of pods and rolling updates. A Service gives pods a stable network endpoint and load balancing.

Q407. What are ConfigMaps and Secrets in Kubernetes?

Answer: ConfigMaps store non-sensitive configuration. Secrets store sensitive data like passwords and API keys. Both are injected into pods as environment variables or files.

Q408. What is kubectl?

Answer: The Kubernetes command-line tool: `kubectl get pods`, `kubectl apply -f deployment.yaml`, `kubectl logs`, `kubectl describe` and `kubectl scale`.

Q409. What is horizontal scaling vs vertical scaling?

Answer: Horizontal scaling adds more instances or machines. Vertical scaling adds more CPU or RAM to an existing machine. Cloud-native apps prefer horizontal scaling.

Q410. What is a rolling deployment? What is blue-green deployment?

Answer: A rolling deployment gradually replaces old instances with new ones. Blue-green runs two identical environments and switches traffic to the new one, allowing instant rollback.

Q411. What is cloud computing?

Answer: On-demand delivery of computing resources (servers, storage, databases, AI) over the internet with pay-as-you-go pricing, like AWS, Azure and Google Cloud.

Q412. What are IaaS, PaaS and SaaS?

Answer: IaaS provides virtual machines and infrastructure (AWS EC2, Azure VMs). PaaS provides a platform to deploy apps without managing servers (Azure App Service). SaaS is ready-to-use software (Microsoft 365, Gmail).

Q413. What is AWS EC2?

Answer: Elastic Compute Cloud provides resizable virtual servers (instances) in AWS. You choose instance types, OS, security groups and key pairs, and pay for usage.

Q414. How would you deploy an ASP.NET Core app on EC2?

Answer: Launch an instance, configure security groups (open ports 80/443), install the .NET runtime (or Docker), copy the published app or pull the image, run it with Kestrel behind Nginx as a reverse proxy, and configure a service so it restarts automatically.

Q415. What is a security group in AWS?

Answer: A virtual firewall for EC2 instances that controls inbound and outbound traffic by port, protocol and source IP.

Q416. What are the main Azure services relevant to .NET developers?

Answer: Azure App Service (web apps and APIs), Azure Functions (serverless), Azure SQL Database, Azure Storage, Key Vault (secrets), Azure OpenAI, Azure AI Search, AKS (Kubernetes) and Application Insights (monitoring).

Q417. What is serverless computing?

Answer: Running code without managing servers. The cloud provider handles scaling, and you pay only for execution. Examples: Azure Functions and AWS Lambda.

Q418. What is Azure Key Vault?

Answer: A service to securely store and manage secrets, keys and certificates. Apps access it through managed identities instead of storing credentials in code.

Q419. What is a managed identity in Azure?

Answer: An automatically managed identity in Microsoft Entra ID (Azure AD) that lets Azure resources authenticate to other services (like Key Vault or Azure OpenAI) without storing credentials.

Q420. What is monitoring and logging in production?

Answer: Tracking app health, performance and errors with tools like Azure Application Insights, CloudWatch, Prometheus and Grafana, and setting alerts to respond quickly to issues.

Q421. What is Infrastructure as Code?

Answer: Managing infrastructure through code files instead of manual setup. Examples: Terraform, ARM or Bicep templates, AWS CloudFormation. It's repeatable, versioned and automated.

Q422. What does the Google Cloud Practitioner certification cover?

Answer: Foundational cloud concepts: cloud service models, core Google Cloud services (compute, storage, databases, networking), security basics, pricing and billing, and the business value of cloud.

Q423. What is a microservices architecture?

Answer: An architecture where an app is split into small, independent services that each own a business capability and communicate through APIs or messaging. They can be deployed and scaled independently, but add complexity.

Q424. What is the difference between a monolith and microservices?

Answer: A monolith is one deployable unit: simple to start, but harder to scale and change as it grows. Microservices are independent services: flexible and scalable, but with more operational complexity.

Python, Machine Learning & Deep Learning

Q425. Why is Python popular for ML?

Answer: It has simple syntax, a huge ecosystem (NumPy, Pandas, Scikit-learn, TensorFlow, PyTorch), strong community support and quick prototyping.

Q426. What is the difference between a list and a tuple in Python?

Answer: Lists are mutable (can change) and use []. Tuples are immutable, use () and are slightly faster. Tuples can be dictionary keys.

Q427. What is the difference between a list and a NumPy array?

Answer: NumPy arrays store homogeneous data in contiguous memory and support fast vectorized operations. Lists are flexible but much slower for numerical computation.

Q428. What is Pandas?

Answer: A Python library for data manipulation and analysis. Its main structures are Series (1D) and DataFrame (2D table), with functions to clean, filter, group, merge and analyze data.

Q429. How do you handle missing values in Pandas?

Answer: Detect them with `isnull()`. Drop them with `dropna()`, or fill them with `fillna()` using the mean, median, mode or a constant, depending on the data and context.

Q430. What is Seaborn?

Answer: A Python visualization library built on Matplotlib for statistical plots like heatmaps, count plots and distribution plots. I used it for data analysis and visualizing model performance, such as confusion matrices.

Q431. What is Machine Learning?

Answer: A branch of AI where systems learn patterns from data to make predictions or decisions without being explicitly programmed with rules.

Q432. What are supervised, unsupervised and reinforcement learning?

Answer: Supervised: learns from labeled data (classification, regression). Unsupervised: finds patterns in unlabeled data (clustering, dimensionality reduction). Reinforcement: an agent learns by trial and error through rewards.

Q433. What is the difference between classification and regression?

Answer: Classification predicts discrete classes (like which sign gesture). Regression predicts continuous values (like a stock price).

Q434. Name some common ML algorithms.

Answer: Linear regression, logistic regression, decision trees, random forest, SVM, KNN, Naive Bayes, K-Means (clustering), PCA (dimensionality reduction) and gradient boosting (XGBoost).

Q435. What is overfitting and underfitting?

Answer: Overfitting: the model memorizes training data, including noise, and performs poorly on new data. Underfitting: the model is too simple to capture patterns and performs poorly everywhere.

Q436. How do you prevent overfitting?

Answer: More data, data augmentation, regularization (L1/L2), dropout, early stopping, simpler models, cross-validation and removing irrelevant features.

Q437. What is the bias-variance trade-off?

Answer: High bias (a model that's too simple) causes underfitting. High variance (a model that's too complex) causes overfitting. The goal is a balance that generalizes well.

Q438. What is train/validation/test split?

Answer: Training data teaches the model. Validation data tunes hyperparameters and checks overfitting. Test data gives a final unbiased performance estimate. A common split is 70/15/15.

Q439. What is cross-validation?

Answer: Splitting data into k folds, training on k-1 folds and validating on the remaining fold, repeated k times. It gives a more reliable performance estimate.

Q440. What evaluation metrics are used for classification?

Answer: Accuracy, precision (of predicted positives, how many are correct), recall (of actual positives, how many are found), F1-score (harmonic mean of precision and recall) and the confusion matrix.

Q441. What is a confusion matrix?

Answer: A table comparing actual and predicted classes: true positives, true negatives, false positives and false negatives. For multi-class problems like sign language, it shows which gestures get confused with each other.

Q442. When is accuracy not a good metric?

Answer: With imbalanced data. If 95% of samples are one class, a model predicting only that class gets 95% accuracy but is useless. Use precision, recall and F1 instead.

Q443. What is feature engineering?

Answer: Creating, transforming or selecting input features to improve model performance: scaling, encoding categories, creating new features and removing irrelevant ones.

Q444. What is normalization vs standardization?

Answer: Normalization scales values to a range like 0 to 1 (min-max). Standardization rescales to mean 0 and standard deviation 1. In images, dividing pixel values by 255 is normalization.

Q445. What is Scikit-learn?

Answer: A Python ML library with tools for preprocessing, classical ML algorithms, model selection (train_test_split, GridSearchCV) and evaluation metrics.

Q446. What is Deep Learning?

Answer: A subset of ML that uses multi-layered neural networks to learn complex patterns automatically from large data. It powers image recognition, NLP and GenAI.

Q447. What is an Artificial Neural Network?

Answer: A model of interconnected layers of neurons (input, hidden and output). Each neuron applies weights, a bias and an activation function. It learns by adjusting weights through backpropagation.

Q448. What is a Convolutional Neural Network?

Answer: A neural network designed for images. Convolutional layers use filters to detect features (edges, shapes), pooling layers reduce dimensions, and fully connected layers classify. It captures spatial patterns efficiently.

Q449. Why did CNN work better than ANN for sign language detection?

Answer: An ANN flattens the image and loses spatial relationships between pixels. A CNN uses convolution filters that learn local features like finger edges and hand shapes, shares weights (fewer parameters) and is more robust to position changes, so it generalizes better on images.

Q450. What are the layers of a CNN?

Answer: Input, convolution (feature extraction with filters), activation (ReLU), pooling (downsampling), flatten, fully connected (dense) and output (softmax for multi-class classification).

Q451. What is pooling, and why is it used?

Answer: Pooling (usually max pooling) reduces the spatial size of feature maps. That lowers computation, controls overfitting and makes the features more robust to small shifts.

Q452. What are activation functions?

Answer: Functions that add non-linearity so networks can learn complex patterns. ReLU is common in hidden layers. Sigmoid is used for binary output. Softmax is used for multi-class output, as in sign classification.

Q453. What is backpropagation?

Answer: The algorithm that computes gradients of the loss with respect to each weight using the chain rule, from output back to input, so the weights can be updated to reduce error.

Q454. What is gradient descent?

Answer: An optimization algorithm that updates weights in the direction that reduces the loss, step by step, with a learning rate. Variants include SGD, mini-batch and Adam.

Q455. What is the learning rate?

Answer: The step size for weight updates. Too high and training diverges or oscillates. Too low and training is very slow.

Q456. What is an epoch, a batch and an iteration?

Answer: An epoch is one full pass over the training data. A batch is a subset of samples processed together. An iteration is one weight update, done on one batch.

Q457. What is dropout?

Answer: A regularization technique that randomly turns off a fraction of neurons during training, which prevents overfitting by reducing reliance on specific neurons.

Q458. What is data augmentation?

Answer: Creating modified versions of training images through rotation, flipping, zooming and brightness changes. It increases dataset variety and reduces overfitting. I used it in sign language detection to improve generalization.

Q459. Explain your Sign Language Detection project.

Answer: I built a system that recognizes hand gestures in images and classifies them into sign language characters. I preprocessed the images (resizing, normalization, augmentation), trained and compared ANN and CNN models, and used Pandas and Seaborn to analyze data and visualize performance, including confusion matrices, to guide improvements.

Q460. What challenges did you face in the sign language project?

Answer: (*Customize.*) Common ones: similar-looking gestures getting confused, variations in lighting, background and hand position, and limited data. I addressed them with preprocessing, augmentation and a CNN architecture, and used confusion matrices to find the problem classes.

Q461. How would you make sign language detection work in real time?

Answer: Capture webcam frames with OpenCV, detect and crop the hand (for example with MediaPipe), preprocess each frame the same way as the training data, run the trained CNN, and smooth predictions over several frames for stability.

Q462. What is transfer learning?

Answer: Using a model pre-trained on a large dataset (like ResNet or MobileNet on ImageNet) and fine-tuning it on your smaller dataset. It gives better accuracy with less data and training time.

Q463. What is the Stock Prediction project from your ML certification?

Answer: A deep learning model that predicts stock prices from historical data, typically using time-series models like LSTM. It involved scaling data, creating sequences of past prices, training, and evaluating predictions against actual values with metrics like RMSE.

Q464. What is an LSTM?

Answer: Long Short-Term Memory is a type of recurrent neural network with gates (input, forget, output) that can remember long-term dependencies in sequences. It's used for time series and text.

Q465. What is K-Means clustering?

Answer: An unsupervised algorithm that groups data into K clusters by assigning points to the nearest centroid and updating centroids iteratively until they stabilize.

Project Deep-Dive, SDLC, Agile & Problem Solving

Q466. Explain your Online Banking Application end to end.

Answer: A full-stack app with an ASP.NET Core Web API backend and an Angular front end. Users log in with role-based access for customers and managers. Customers can create accounts, deposit, withdraw, transfer funds and view transaction history on a dashboard. The backend uses a layered architecture (controllers, services, repositories) with a SQL database, validation, error handling and NUnit tests for business logic.

Q467. How did you implement fund transfers safely?

Answer: Validate the amount and the accounts, check sufficient balance, then debit the source and credit the destination inside a single database transaction. If any step fails, roll back, and record the transaction with its status for audit.

Q468. How do you prevent a negative balance or double withdrawal under concurrency?

Answer: Use database transactions with appropriate isolation, row locking or optimistic concurrency (a rowversion column in EF Core), and a CHECK constraint ($\text{Balance} \geq 0$) as a final safeguard.

Q469. How did you store passwords securely?

Answer: Never store plain text. Hash with a strong, salted, slow algorithm like BCrypt, PBKDF2 (ASP.NET Identity's default) or Argon2, and compare hashes at login.

Q470. What challenges did you face in the banking project?

Answer: (*Customize.*) Examples: configuring CORS between Angular and the API, managing JWT and role-based access, keeping transfers consistent with transactions, and structuring code into layers for testability.

Q471. If your banking app had 1 million users, what would you change?

Answer: Add caching (Redis), database indexing and read replicas, run stateless API instances behind a load balancer with horizontal scaling, use async processing (queues) for heavy tasks, rate limiting, CDN for the front end, monitoring, and possibly split into microservices.

Q472. What is the SDLC?

Answer: The Software Development Life Cycle: requirement analysis, planning, design, development, testing, deployment and maintenance.

Q473. What is the difference between the Waterfall and Agile models?

Answer: Waterfall is sequential, with each phase finishing before the next, and suits fixed requirements. Agile is iterative, delivering in short sprints with continuous feedback, and suits changing requirements.

Q474. What is Scrum?

Answer: An Agile framework with roles (Product Owner, Scrum Master, Development Team), events (Sprint Planning, Daily Stand-up, Sprint Review, Retrospective) and artifacts (Product Backlog, Sprint Backlog, Increment).

Q475. What happens in a daily stand-up?

Answer: A short meeting (about 15 minutes) where each member shares what they did yesterday, what they'll do today, and any blockers.

Q476. What is Jira and how have you used it?

Answer: Jira is Atlassian's project tracking tool for Agile teams. I've used it to track tasks and bugs, manage sprint boards, update ticket status, log work and prioritize items, including my automation tasks.

Q477. What are Epics, Stories, Tasks and Bugs in Jira?

Answer: An Epic is a large body of work. A Story is a user-focused requirement. A Task is a unit of work. A Bug is a defect. Epics contain stories, which can have subtasks.

Q478. What are story points?

Answer: A relative estimate of effort, complexity and uncertainty for a user story, often on the Fibonacci scale (1, 2, 3, 5, 8). They help plan sprint capacity.

Q479. What is a sprint retrospective?

Answer: A meeting at the end of a sprint where the team discusses what went well, what didn't, and what to improve next sprint.

Q480. What is Big O notation?

Answer: A way to describe how an algorithm's time or space grows with input size. For example, $O(1)$ is constant, $O(\log n)$ is binary search, $O(n)$ is a linear scan, $O(n \log n)$ is efficient sorting, and $O(n^2)$ is nested loops.

Q481. What is the difference between an array and a linked list?

Answer: Arrays have contiguous memory and $O(1)$ index access, but costly inserts and deletes in the middle. Linked lists have nodes with pointers, $O(1)$ insert or delete at a known position, but $O(n)$ access.

Q482. What is the difference between a stack and a queue?

Answer: A stack is LIFO (push, pop), used for undo and recursion. A queue is FIFO (enqueue, dequeue), used for task scheduling and BFS.

Q483. What is a hash table? What is its time complexity?

Answer: A structure that maps keys to values using a hash function. Insert, delete and lookup are $O(1)$ on average and $O(n)$ in the worst case with many collisions. In C#, it's `Dictionary<TKey,TValue>`.

Q484. What is binary search?

Answer: An $O(\log n)$ search on a sorted array that repeatedly compares the middle element and halves the search space.

Q485. Name common sorting algorithms and their complexities.

Answer: Bubble, selection and insertion sort: $O(n^2)$. Merge sort: $O(n \log n)$, stable. Quick sort: $O(n \log n)$ average, $O(n^2)$ worst. Heap sort: $O(n \log n)$.

Q486. How do you reverse a string in C#?

Answer: `new string(str.Reverse().ToArray())`, or convert to a char array, use `Array.Reverse`, and create a new string. Or use two pointers swapping characters from both ends, which is $O(n)$.

Q487. How do you check if a string is a palindrome?

Answer: Use two pointers, one at the start and one at the end, comparing characters while moving inward. If all pairs match, it's a palindrome. $O(n)$ time, $O(1)$ space.

Q488. How do you find duplicates in an array efficiently?

Answer: Iterate while adding elements to a HashSet. If an add fails, the element is a duplicate. $O(n)$ time, $O(n)$ space.

Q489. Solve the Two Sum problem.

Answer: Use a Dictionary of value to index. For each number, check whether target minus the number exists in the dictionary. If yes, return both indices; otherwise add the current number. $O(n)$ time.

Q490. How do you find the first non-repeating character in a string?

Answer: Count frequencies with a Dictionary in one pass, then scan the string again and return the first character with count 1. $O(n)$.

Q491. What is recursion? What are its risks?

Answer: A function calling itself with a smaller problem until it reaches a base case. Risks are stack overflow from deep recursion and repeated work, which memoization or iteration can fix.

Q492. What is the difference between BFS and DFS?

Answer: BFS explores level by level using a queue and finds the shortest path in unweighted graphs. DFS goes deep first using a stack or recursion, and is useful for path finding and cycle detection.

Q493. How do you approach a problem you've never seen before in an interview?

Answer: Clarify the requirements and edge cases, discuss a brute-force solution, then optimize with suitable data structures, explain the time and space complexity, write clean code, and test with examples, including edge cases.

Q494. What is the difference between a process and a thread?

Answer: A process is an independent program with its own memory space. A thread is a lightweight unit of execution inside a process that shares the process's memory. Threads are cheaper to create but need synchronization.

Q495. What is a race condition?

Answer: When multiple threads access shared data at the same time and the result depends on timing. Prevent it with locks, thread-safe collections or immutable data.

Q496. What is a load balancer?

Answer: A component that distributes incoming traffic across multiple servers to improve availability, performance and fault tolerance.

Q497. What is the difference between authentication with sessions and with tokens?

Answer: Session-based auth stores state on the server and uses a session cookie. Token-based auth (JWT) is stateless: the client sends a signed token with each request, which scales better for APIs and SPAs.

Q498. What is a message queue and why use one?

Answer: A queue (Azure Service Bus, RabbitMQ, Kafka) lets services communicate asynchronously. It decouples components, handles traffic spikes and improves reliability for background tasks.

Q499. What is rate limiting?

Answer: Restricting how many requests a client can make in a time window, to protect APIs from abuse and overload. ASP.NET Core has built-in rate limiting middleware.

Q500. How would you design a URL shortener at a high level?

Answer: An API accepts a long URL, generates a unique short code (base62 of an ID or a hash), stores the mapping in a database, and redirects GET /{code} to the original URL. Add caching for popular links, indexing on the code and rate limiting.